

Mémoire d'ingénieur

Industrialisation de la chaîne de valeur d'une ESN

David Guichard

Année 2025–2026

Stage de fin d'études réalisé dans l'entreprise

Oxyl

en vue de l'obtention du diplôme d'ingénieur de TELECOM Nancy

Maître de stage : Maxime Bauer

Encadrant universitaire : Thibault Cholez

Déclaration sur l'honneur de non-plagiat

Je soussigné(e),

Nom, prénom : Guichard, David

Élève-ingénieur(e) régulièrement inscrit(e) en 3^e année à TELECOM Nancy

Numéro de carte de l'étudiant(e) : 322119809

Année universitaire : 2025–2026

Auteur(e) du document, mémoire, rapport ou code informatique intitulé :

Industrialisation de la chaîne de valeur d'une ESN : développement
du capital humain et innovation technologique

Par la présente, je déclare m'être informé(e) sur les différentes formes de plagiat existantes et sur les techniques et normes de citation et référence.

Je déclare en outre que le travail rendu est un travail original, issu de ma réflexion personnelle, et qu'il a été rédigé entièrement par mes soins. J'affirme n'avoir ni contrefait, ni falsifié, ni copié tout ou partie de l'œuvre d'autrui, en particulier texte ou code informatique, dans le but de me l'accaparer.

Je certifie donc que toutes formulations, idées, recherches, raisonnements, analyses, programmes, schémas ou autre créations, figurant dans le document et empruntés à un tiers, sont clairement signalés comme tels, selon les usages en vigueur.

Je suis conscient(e) que le fait de ne pas citer une source ou de ne pas la citer clairement et complètement est constitutif de plagiat, que le plagiat est considéré comme une faute grave au sein de l'Université, et qu'en cas de manquement aux règles en la matière, j'encourrais des poursuites non seulement devant la commission de discipline de l'établissement mais également devant les tribunaux de la République Française.

Fait à Nancy, le 31 août 2026

Signature :



Attestation de validation du mémoire par l'organisme d'accueil

Je soussigné(e),

Nom, prénom : Maxime BAUER

Qualité / Fonction : Tuteur industriel & Développeur Senior

Entreprise d'accueil : Oxyl (19-21, rue du 8 Mai 1945, 94110 Arcueil)

atteste par la présente avoir pris connaissance de la version finale du mémoire d'ingénieur intitulé :

**« Industrialisation de la chaîne de valeur d'une ESN : développement
du capital humain et innovation technologique »**

rédigé par l'élève-ingénieur **David GUICHARD** dans le cadre de son Projet de Fin d'Études (3^e année) en vue de l'obtention du diplôme d'ingénieur de **TELECOM Nancy**.

Je certifie que :

- Les travaux, méthodologies et résultats présentés sont conformes aux réalisations menées au sein d'Oxyl;
- Le document ne divulgue aucun secret d'affaires, procédé brevetable non protégé ou information confidentielle non autorisée;
- J'autorise la diffusion et la communication du présent mémoire aux membres du jury de soutenance de TELECOM Nancy.

Fait à : Arcueil

Le : 18 août 2026

Cachet de l'entreprise :

OXYL GROUPE
19-21 rue du 8 Mai 1945
94110 ARCUEIL
Siren : 844 849 042
contact@oxylgroupe.fr

Signature de l'encadrant industriel :



.....

.....

Mémoire d'ingénieur

Industrialisation de la chaîne de valeur d'une ESN

David Guichard

Année 2025–2026

Stage de fin d'études réalisé dans l'entreprise Oxyl
en vue de l'obtention du diplôme d'ingénieur de TELECOM Nancy

David Guichard
5, rue Pierre d'Artagnan
92350 Le Plessis-Robinson
06 20 32 37 21
david.guichard@telecomnancy.eu

TELECOM Nancy
193 avenue Paul Muller,
CS 90172, VILLERS-LÈS-NANCY
+33 (0)3 83 68 26 00
contact@telecomnancy.eu

Oxyl
19-21, rue du 8 Mai 1945
94110 Arcueil
09 72 60 09 87



Maître de stage : Maxime Bauer

Encadrant universitaire : Thibault Cholez

Remerciements

Je tiens à exprimer ma profonde gratitude à l'ensemble des personnes qui ont contribué, par leur encadrement, leurs conseils avisés et leur soutien, au bon déroulement et à la réussite de ce Projet de Fin d'Études au sein du Groupe Oxyl.

Mes remerciements s'adressent tout particulièrement à :

- **Maxime BAUER**, mon tuteur de stage et reviewer senior, pour sa disponibilité constante, sa bienveillance et l'exigence constructive de ses revues de code lors des sprints du backend de la plateforme de quiz. Ses précieux retours d'ingénierie et sa vision logicielle ont grandement enrichi ma pratique du développement ;
- **Selena HIND WOODWARD**, Directrice des Ressources Humaines d'Oxyl, pour sa confiance accordée dès le processus de recrutement, son accueil chaleureux et pour ses enseignements particulièrement formateurs sur les méthodes agiles et la dynamique collaborative au sein de l'entreprise ;
- **Clément DUBOIS**, pour son coaching technique rigoureux lors des sprints backend et ses revues de code pointues, qui ont constitué un moteur constant d'apprentissage et d'élévation des standards de qualité ;
- **Jeremy ROCA**, développeur senior et chef d'équipe sur le projet de l'usine logicielle agentic, pour la qualité de son accueil, son *onboarding* méthodique et sa pédagogie dans la transmission des enjeux d'architecture du moteur ;
- **Jordan PETER** et **Alexandre DANTY**, développeurs seniors et relais d'équipe au sein du projet de l'usine logicielle, pour leur disponibilité, leur écoute et leurs conseils techniques pertinents lors des phases de conception et de synchronisation d'équipe ;
- **Mathéo ALLARD**, Project Manager des projets internes, pour son pilotage précieux et ses orientations claires sur les attendus du projet d'usine logicielle, ainsi que pour ses conseils avisés sur les bonnes pratiques de revue de code et le déroulement des *daily*s ;
- **Pierre VALAT**, collègue stagiaire et compagnon de route sur l'ensemble de ce parcours – de la plateforme de quiz jusqu'à l'usine logicielle – pour cette collaboration quotidienne stimulante, l'entraide technique et la dynamique d'équipe partagée ;
- **Thibault CHOLEZ**, mon enseignant-référent et tuteur académique à TELECOM Nancy, pour son suivi attentif, sa disponibilité et ses conseils méthodologiques tout au long de ce projet de fin d'études.

Enfin, j'adresse mes sincères remerciements à l'ensemble des collaborateurs du Groupe Oxyl pour leur accueil et la richesse des échanges au quotidien, au corps professoral et à l'équipe pédagogique de TELECOM Nancy pour la solidité de la formation délivrée, ainsi qu'à ma famille et mes proches pour leur soutien indéfectible tout au long de mon cursus d'ingénieur.

Avant-propos

Ce Projet de Fin d'Études chez Oxyl concrétise mon parcours à TELECOM Nancy et mon projet professionnel.

Durant mon cursus, j'ai choisi la filière *Logiciel Embarqué* avec l'objectif d'acquérir de solides bases en programmation bas niveau et en architecture logicielle, indispensables pour concevoir des systèmes fiables et performants.

Ce lien entre socle logiciel et intelligence artificielle s'est précisé lors de mon stage de deuxième année au laboratoire CRAN, où j'ai développé une architecture en *Deep Learning* pour un bras robotique en cobotique industrielle. J'ai ensuite approfondi le *Machine Learning* pendant mon semestre académique à Madrid. Mon arrivée chez Oxyl m'a permis d'associer cette double culture pour contribuer au développement d'une usine logicielle automatisée par des agents IA.

À l'avenir, je souhaite poursuivre dans l'automatisation des processus par des systèmes agentiques, avec la conviction que l'efficacité de l'IA repose avant tout sur la qualité et la rigueur du socle logiciel sous-jacent.

David GUICHARD
Le Plessis-Robinson, le 18 août 2026

Table des matières

Remerciements	vii
Avant-propos	ix
Table des matières	xi
1 Introduction Générale	1
2 Vers une industrialisation de la chaîne de valeur (Le cadre)	5
2.1 L'ESN comme « usine » de talent et de code : Présentation d'Oxyl	5
2.1.1 Présentation du Groupe Oxyl	5
2.1.2 Organisation et culture d'entreprise	6
2.1.3 L'« usine de talents » : Recrutement et formation certifiante	6
2.1.4 Cycle de vie du collaborateur et formation continue	7
2.2 Problématique : Concilier rigueur humaine et automatisation par l'IA	8
2.2.1 La divergence apparente : Artisanat technique contre automatisation par l'IA	8
2.2.2 La problématique centrale et la réponse par la synergie	8
3 La standardisation du capital humain (Volet 1 - Le Quiz)	11
3.1 Analyse des besoins : Le rôle du socle Java/JEE dans la formation	11
3.1.1 L'intérêt des certifications industrielles	11
3.1.2 Le choix de Java et de la POO comme socle technique	11
3.1.3 Limites de l'outil existant et besoins identifiés	12
3.1.4 Cahier des charges et fonctionnalités de NewroFactory	12
3.1.5 Objectifs de réalisation	14
3.2 Conception et architecture de la plateforme	14
3.2.1 Modélisation des données et arborescence des chapitres	14
3.2.2 Architecture logicielle backend (Spring)	16
3.2.3 Sécurité et gestion des accès	18

3.2.4	Développement du frontend sous Angular	19
3.2.5	Organisation d'équipe et retours d'expérience	21
3.3	Justification des choix techniques	21
3.3.1	Le choix de Spring Core et de la configuration explicite	21
3.3.2	Le choix d'Angular pour le frontend	22
3.3.3	Monolithe modulaire vs Microservices	22
3.3.4	Outils et bibliothèques complémentaires	22
4	La standardisation du capital technique (Volet 2 - L'usine IA)	23
4.1	État de l'art : Industrialisation du SDLC et agents IA (SWE-bench)	23
4.1.1	L'évolution du SDLC et la modification de code	23
4.1.2	Évaluation par SWE-bench et assistants en ligne de commande (CLI) . . .	23
4.1.3	Comparatif des solutions agentiques du marché	24
4.2	Théorie de l'abstraction : Patrons de conception contre le lock-in	24
4.2.1	Choix et rôles des patrons de conception	24
4.2.2	Modélisation de la couche d'abstraction des LLMs	25
4.2.3	Défis techniques d'intégration en conteneur	25
4.3	Gestion des contraintes opérationnelles : Coûts, sécurité et garde-fous	26
4.3.1	Maîtrise des coûts de tokens : Limites de réessais et disjoncteur	26
4.3.2	Gestion du contexte et rôle du fichier Manifest	26
4.3.3	Sécurité et protection des secrets	27
4.3.4	Vérifications mécaniques et garde-fous	27
4.3.5	Observabilité et perspectives	27
5	La mise en œuvre de l'infrastructure agnostique	29
5.1	Architecture du moteur agnostique : Découplage de Git et des LLMs	29
5.1.1	Contrats d'interface : BacklogProvider et CodeBaseProvider	29
5.1.2	Types étiquetés (<i>Branded Types</i>) et résolution dynamique	30
5.1.3	Cycle opérationnel : Isolation par <i>Git Worktrees</i>	30
5.1.4	Exécution du pipeline et publication de la Merge Request	30
5.1.5	Schéma d'architecture globale du moteur	31
5.2	Stratégies de normalisation des forges et des agents	31
5.2.1	Modèle pivot canonique : Unification des tickets et MRs	31
5.2.2	Utilisation des CLI agentiques autonomes	32

5.2.3	Échange documentaire via les fichiers Markdown (<code>.task/* .md</code>)	33
5.3	Validation et retours d'expérience sur cas réels	33
5.3.1	Expérimentations sur projets réels chez Oxyl	33
5.3.2	Étude de cas : Résolution du Ticket #17	33
5.3.3	Le modèle Human-in-the-Loop et critères d'évaluation des LLMs	34
6	Analyse critique et impact organisationnel	37
6.1	Synergie entre formation humaine et usine logicielle	37
6.1.1	Déplacement du travail et de la charge cognitive	37
6.1.2	L'importance des fondamentaux théoriques pour relire le code de l'IA . .	37
6.1.3	Transversalité des concepts : de Java à TypeScript	38
6.1.4	Boucle d'apprentissage continu : Synergie MNF et Usine IA	38
6.2	Impact économique, productivité et anti-lock-in	38
6.2.1	Retour sur investissement (ROI) et gains de temps	38
6.2.2	Optimisation des coûts d'API et indépendance technologique	39
6.2.3	Valorisation de la formation et réduction de la dette technique	39
6.3	Limites techniques, sécurité et impacts humains	39
6.3.1	Limites des modèles et importance de l'atomicité des tickets	39
6.3.2	Sécurité et confidentialité des données	40
6.3.3	Évolution du rôle du développeur et formation des juniors	40
7	Conclusion Générale	41
	Bibliographie / Webographie	43
	Liste des illustrations	45
	Liste des tableaux	47
	Listings	49
	Glossaire	52
	Annexes	54
A	Attestations des certifications obtenues	55
A.1	Oracle Certified Associate (OCA) – Java SE 8 Programmer	55

A.2	AWS Certified Cloud Practitioner	57
B	Diagrammes de classe et Schémas d'architecture logicielle	59
B.1	Plateforme d'Évaluation : Modèle du Domaine Métier (Backend Spring)	59
B.2	Plateforme d'Évaluation : Architecture Logicielle en Couches et Aspects (Spring) .	61
B.3	Plateforme d'Évaluation : Architecture Frontend Réactive (Angular)	63
B.4	Usine Logicielle : Patrons de Conception et Abstractions des Fournisseurs	65
B.5	Usine Logicielle : Moteurs de Validation Mécanique et Cycle du Worker	67
C	Exemples de traces d'exécution de l'usine logicielle	69
C.1	Journal brut d'exécution (Format JSON Lines)	69
C.2	Analyse détaillée des jalons de l'exécution	71
D	Vues d'écrans et Interfaces complémentaires de la plateforme NewroFactory	73
D.1	Authentification et Contrôle d'accès	73
D.2	Tableau de bord général de supervision administrateur	73
D.3	Référentiel des compétences et modules de formation	74
D.4	Historique global et filtrage multicritères des séries	74
	Résumé	77
	Abstract	77

1 Introduction Générale

Contexte sectoriel : L'évolution du modèle des ESN

Le secteur des Entreprises de Services du Numérique (ESN) évolue rapidement avec l'essor de l'intelligence artificielle générative. Historiquement, les ESN fonctionnaient principalement en régie, avec une facturation au temps passé. Aujourd'hui, les clients demandent de plus en plus des projets au forfait avec des engagements stricts de délais et de résultats. Dans ce cadre, la maîtrise des coûts et la productivité deviennent décisives.

En parallèle, l'arrivée des assistants de code modifie le travail des développeurs. Les tâches de base souvent confiées aux profils juniors sont désormais partiellement automatisées. De plus, les exigences des clients sont contrastées : certains souhaitent accélérer leurs développements grâce à l'IA, tandis que d'autres (banque, défense, secteur public) interdisent l'envoi de code vers des API externes pour des raisons de confidentialité et de souveraineté. Pour une ESN, le défi consiste donc à garantir une excellente qualité technique humaine tout en automatisant ce qui peut l'être de manière sécurisée.

L'écosystème Oxyl et sa stratégie d'innovation

Dans ce contexte, Oxyl a choisi d'intégrer très tôt l'intelligence artificielle dans ses méthodes de développement. L'objectif d'Oxyl est double : développer une expertise de pointe sur les systèmes agentiques pour concevoir de nouveaux outils, et améliorer l'efficacité de ses consultants sur leurs projets.

Pour soutenir cette démarche, Oxyl recrute principalement des élèves-ingénieurs en stage de fin d'études et leur propose un parcours intensif de formation technique certifiante, leur permettant d'être rapidement opérationnels en clientèle grand compte.

Le double défi d'ingénierie : Compétences humaines et usine logicielle

Le travail réalisé durant ce stage s'articule autour de deux axes complémentaires :

1. **La standardisation des compétences (Volet 1 - Le Quiz) :** Pour répondre aux attentes des clients, les compétences techniques des consultants doivent être solides et certifiées. Oxyl prépare ses recrues aux certifications Oracle (OCA et OCP Java SE [2]) et AWS

Certified Cloud Practitioner. Pour faciliter cette préparation, nous avons développé une plateforme web sur-mesure en Spring Core [8] et Angular. Elle sert à la fois de projet pratique d'apprentissage pour les stagiaires et d'outil quotidien d'entraînement et de suivi pédagogique pour l'entreprise.

2. **L'industrialisation du capital technique (Volet 2 - L'Usine Logicielle)** : Pour automatiser le traitement des tickets récurrents (corrections de bugs, nouvelles fonctionnalités, tests), Oxyl développe une « Usine Logicielle » agentique en TypeScript. Face aux coûts des API de LLM et aux exigences de confidentialité des clients, un impératif d'architecture s'est imposé : rendre le moteur totalement indépendant (*agnostique*) des fournisseurs de modèles (OpenAI, Claude, Google, modèles locaux) et des forges logicielles (GitLab, GitHub, Gitea, Jira, Bitbucket).

Problématique du Projet de Fin d'Études

L'articulation entre ces deux besoins conduit à la problématique suivante :

Dans un contexte de transition vers les projets au forfait et d'automatisation par l'IA, comment concilier la montée en compétences des ingénieurs (garantissant la maîtrise des fondamentaux) et le développement d'une usine logicielle agentique agnostique pour assurer la rentabilité et l'indépendance technologique de l'entreprise?

Méthodologie et organisation du travail

Ce travail s'est déroulé au sein de deux cadres méthodologiques distincts :

- **Le projet de la plateforme d'évaluation (Équipe de 4 stagiaires)** : Le projet a débuté avec des sprints Scrum hebdomadaires rythmés par des revues de code exigeantes sur le backend, avant de basculer en flux Kanban pour le développement collaboratif du frontend Angular.
- **Le projet Usine Logicielle (Équipe R&D agile)** : Intégré au sein de l'équipe de développement interne coordonnée par un Project Manager, le travail a suivi des rituels quotidiens (*dailies*) et des revues de code régulières pour maintenir un haut niveau de qualité logicielle.

Contributions personnelles

Mes contributions principales au cours du stage ont porté sur :

- Le développement *full-stack* de la plateforme de quiz (modélisation du domaine en Java, API REST Spring Core, composants et design system Angular);
- La conception et l'implémentation de la couche d'abstraction de l'Usine Logicielle en TypeScript, en particulier les adaptateurs de forges de code (*CodebaseProviders* : GitLab, Gitea, GitHub) et de gestionnaires de tickets (*BacklogProviders* : GitLab Issues, Jira);

- La mise en œuvre des patrons de conception (*Design Patterns*) assurant l’interchangeabilité des composants et la résilience du moteur face aux erreurs d’exécution.

Structure du mémoire

Le mémoire s’organise en cinq chapitres :

- Le **Chapitre 2** présente l’entreprise Oxyl, son modèle d’intégration des consultants, les risques de dépendance technologique (*vendor lock-in*) et la stratégie liant formation et automatisation.
- Le **Chapitre 3** détaille le premier volet : la conception et l’architecture de la plateforme d’évaluation des compétences (Spring Core / Angular).
- Le **Chapitre 4** aborde le second volet : l’état de l’art sur les agents de développement logiciel (SWE-bench), les patrons d’abstraction et la gestion des contraintes (sécurité, coûts des tokens).
- Le **Chapitre 5** décrit la mise en œuvre de l’infrastructure agnostique en TypeScript, les connecteurs de forges et les résultats obtenus sur des cas concrets.
- Le **Chapitre 6** propose un retour d’expérience complet : analyse du retour sur investissement (ROI), évolution du rôle de l’ingénieur, gestion des limites de l’IA et aspects de confidentialité.
- Enfin, la **Conclusion Générale** dresse le bilan du projet et ouvre sur les perspectives du métier d’ingénieur logiciel.

2 Vers une industrialisation de la chaîne de valeur (Le cadre)

2.1 L'ESN comme « usine » de talent et de code : Présentation d'Oxyl

Pour répondre aux exigences de qualité et de vélocité de ses clients, Oxyl s'appuie sur une double démarche : former rapidement des ingénieurs juniors grâce à un parcours certifiant intensif (l'« usine de talents »), et automatiser la production logicielle via des outils agentiques modernes (l'« usine de code »).

2.1.1 Présentation du Groupe Oxyl

Fondée en 1995 sous le nom d'*AlphaCSP* par Fabrice REBY et des ingénieurs de l'École Centrale de Lille, l'entreprise a été pionnière dans l'adoption de Java/J2EE en France et dans la mise en pratique des méthodes agiles (Scrum, eXtreme Programming). Elle a ensuite évolué pour devenir le **Groupe Oxyl**.

Aujourd'hui basée à Arcueil, la société compte environ 200 collaborateurs pour un chiffre d'affaires d'environ 20 millions d'euros en 2024. Oxyl intervient comme cabinet de conseil et ESN spécialisée sur l'ensemble du cycle de développement logiciel (*SDLC*) : du cadrage d'architecture au déploiement Cloud et au maintien en condition opérationnelle (MCO).

Les domaines d'expertise principaux d'Oxyl couvrent :

- **L'écosystème Java / Spring** : architectures microservices, performances transactionnelles, mapping objet-relationnel et sécurité applicative ;
- **L'écosystème JavaScript / TypeScript** : applications web modernes (Angular, React, Node.js) ;
- **L'ingénierie Cloud & DevOps** : conteneurisation (Docker, Kubernetes), Infrastructure as Code (Terraform) et Cloud public (AWS, Azure, GCP) ;
- **L'intelligence artificielle et l'automatisation** : architectures agentiques et intégration de modèles de langage (LLM) dans les flux de développement.

Oxyl se différencie par plusieurs choix structurants :

1. **Le recrutement et l'accélération de jeunes diplômés** : intégrer des stagiaires de fin d'études et leur apporter une formation initiale rigoureuse validée par des certifications reconnues ;
2. **Une culture technique concrète** : accompagnement par des tuteurs seniors, revues de code systématiques, événements internes de partage technique (TechNights) et mise à

disposition de colocations d'intégration pour les stagiaires venant de région ;

3. **L'investissement précoce dans l'IA agentique** : développer des projets internes de R&D pour tester concrètement les outils d'automatisation avant de les déployer en clientèle ;
4. **Une structure à taille humaine** : un ancrage fort en région parisienne et une proximité entre équipes techniques et direction.

Oxyl accompagne des clients variés : grands comptes du secteur bancaire et de l'assurance (Allianz Trade, BPCE - Casden, Société Générale, MyMoneyBank), acteurs industriels (Rexel, Lafarge, Vinci, JCDecaux) et éditeurs de logiciels (Finance Active, PrimaSolutions).

2.1.2 Organisation et culture d'entreprise

Oxyl applique une organisation inspirée des principes de l'entreprise libérée, favorisant l'autonomie, la responsabilisation des développeurs et des circuits de décision courts.

La culture d'entreprise repose sur trois valeurs partagées :

- **La Passion** : recruter des profils motivés par le développement logiciel et les défis techniques ;
- **L'Excellence** : maintenir des standards élevés de qualité de code (*Clean Code* [6], architecture modulaire, tests) ;
- **Le Partage** : encourager la transmission des connaissances entre seniors et juniors et l'entraide d'équipe.

L'organisation s'articule autour d'une entité support (**Sylex**, qui gère l'administration, les RH et le suivi commercial) et d'entités opérationnelles regroupant les consultants par promotion d'arrivée. La direction technique, assurée par Maxime BAUER, pilote la formation des stagiaires et les projets de R&D internes.

2.1.3 L'« usine de talents » : Recrutement et formation certifiante

L'« usine de talents » désigne le processus formalisé par Oxyl pour former rapidement les stagiaires aux standards de développement attendus en mission.

Le processus de sélection

Présente dans plusieurs écoles d'ingénieurs à travers des conférences techniques, Oxyl reçoit plus de 2 000 candidatures par an. Le processus comprend :

1. Un test algorithmique en ligne d'une heure ;
2. Un entretien de motivation et d'évaluation culturelle ;
3. Un test pratique de développement suivi d'un débriefing technique avec un développeur senior ;
4. Un entretien de validation avec la direction.

Chaque année, environ 40 stagiaires sont retenus. J'ai intégré la promotion d'**avril 2026**.

Le programme de formation initiale

Pendant les trois premiers mois de stage, les recrues suivent un parcours intensif en deux volets :

1. La préparation aux certifications industrielles : Chaque stagiaire prépare et passe des certifications officielles reconnues (voir Annexe A) :

- *Oracle Certified Associate (OCA) Java SE 8 Programmer I* : fondamentaux du langage, typage, POO et gestion des exceptions (Figure A.1);
- *Oracle Certified Professional (OCP) Java SE 21 Developer* : programmation fonctionnelle, Streams, concurrence, I/O et modularité;
- *AWS Certified Cloud Practitioner* : concepts fondamentaux du Cloud, sécurité et services managés AWS (Figure A.2).

Cette préparation s'appuie sur la plateforme interne **MyNewroFactory (MNF)**, avec une heure d'entraînement quotidien et des examens blancs réguliers.

2. Le projet d'application « NewroFactory » : En parallèle, chaque stagiaire redéveloppe l'application de formation au cours de 8 sprints. Le projet démarre en Java pur (CLI, JDBC, Servlets) pour comprendre les mécanismes sous-jacents, avant d'intégrer progressivement les couches d'un framework d'entreprise (Spring Core, MVC, AOP, Hibernate/JPA, Spring Security, puis API REST et frontend Angular). Chaque étape fait l'objet d'une revue de code détaillée avec un encadrant senior pour vérifier le respect des bonnes pratiques (*SOLID*, *KISS*, *DRY*).

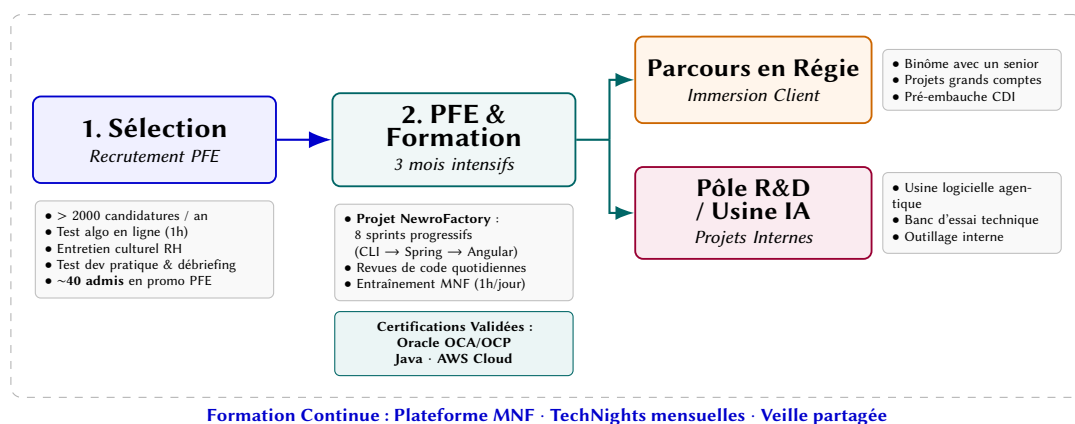


FIGURE 2.1 – Parcours d'intégration et de formation chez Oxyl

2.1.4 Cycle de vie du collaborateur et formation continue

Comme le montre la Figure 2.1, après les trois premiers mois de formation, deux parcours sont possibles :

- **La mission client** : l'ingénieur rejoint un projet en régie, encadré par un consultant senior d'Oxyl pour faciliter son intégration technique et préparer la suite en CDI;
- **L'équipe R&D interne** : l'ingénieur participe au développement des outils internes d'Oxyl (comme l'*Usine Logicielle* agentique).

La montée en compétences se poursuit ensuite grâce à la plateforme MNF, aux conférences internes mensuelles (*TechNights*) et aux retours d'expérience partagés entre équipes.

2.2 Problématique : Concilier rigueur humaine et automatisation par l'IA

L'évolution du marché des ESN vers les contrats au forfait et l'essor des modèles d'IA générative posent un dilemme technique et organisationnel majeur. Deux approches semblent a priori diverger :

2.2.1 La divergence apparente : Artisanat technique contre automatisation par l'IA

D'un côté, le renforcement du **capital humain (Volet 1)** exige un apprentissage exigeant des couches basses : programmation en Java natif sans framework magique, maîtrise de la concurrence, gestion fine de la mémoire JVM et requêtes SQL optimisées. Cette démarche garantit une compréhension intime des systèmes, mais demande du temps.

D'un autre côté, l'**usine logicielle agentique (Volet 2)** recherche l'accélération maximale : délégation de l'écriture du code à des modèles probabilistes (LLMs) pour traiter des tickets de façon asynchrone et réduire les temps de cycle.

Cette opposition apparente soulève deux risques majeurs :

1. **Le risque d'atrophie technique** : en déléguant trop tôt l'écriture de code à l'IA, le développeur junior risque de perdre son sens critique et de valider aveuglément des solutions contenant des failles de sécurité ou des fuites de mémoire invisibles aux tests simples ;
2. **Le risque de verrouillage propriétaire (*vendor lock-in*)** : confier sa production logicielle aux API de grands fournisseurs de Cloud (OpenAI, Anthropic, Google) expose l'entreprise à la volatilité tarifaire et à des contraintes de confidentialité strictes pour les clients du secteur bancaire ou de la défense.

2.2.2 La problématique centrale et la réponse par la synergie

Dès lors, la question centrale de ce projet de fin d'études s'énonce ainsi :

Comment industrialiser la chaîne de production logicielle grâce aux agents d'IA sans atrophier les compétences fondamentales des développeurs juniors ni enfermer l'entreprise dans une dépendance technologique ?

La réponse apportée par Oxyl repose sur la complémentarité directe des deux volets (Figure 2.2) :

- **L'ingénieur comme superviseur et garant de la qualité** : la formation certifiante (OCA/OCP) apporte le recul technique nécessaire pour rédiger des spécifications précises, cadrer les prompts et auditer ligne par ligne le code généré par la machine ;
- **L'usine comme accélérateur agnostique** : construite autour de patrons de conception découplés (*Adapter, Strategy*), l'usine logicielle prend en charge les tâches répétitives (*boilerplate*, squelettes de classes, tests de base) tout en permettant de basculer instantanément d'un modèle d'IA ou d'une forge à une autre selon les contraintes du client.

Le présent mémoire s'articule autour de ces réalisations :

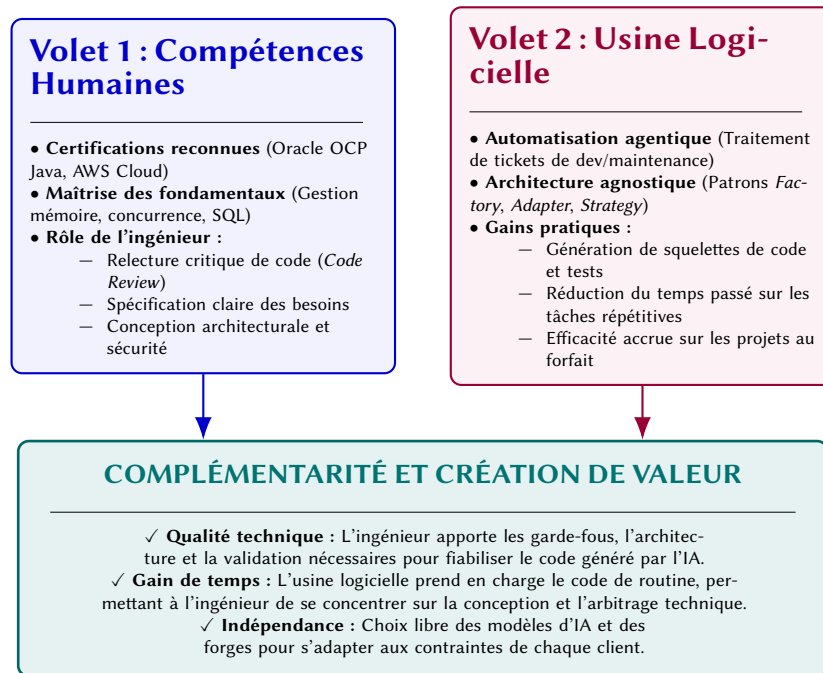


FIGURE 2.2 – Complémentarité entre compétences certifiées et usine logicielle agnostique

- Le **Chapitre 3** détaille la standardisation du capital humain à travers la conception et le développement de la plateforme de formation *NewroFactory* (Volet 1);
- Les **Chapitres 4** et **5** explorent l'industrialisation du capital technique, depuis l'état de l'art agentique jusqu'à la mise en œuvre de l'infrastructure logicielle agnostique (Volet 2);
- Enfin, le **Chapitre 6** propose une analyse critique systémique de cette synergie, mesurant ses retombées industrielles, ses gains de productivité et ses limites éthiques.

3 La standardisation du capital humain (Volet 1 - Le Quiz)

3.1 Analyse des besoins : Le rôle du socle Java/JEE dans la formation

Pour une ESN, la standardisation des compétences consiste à s'assurer que chaque consultant maîtrise les fondamentaux techniques attendus par les clients. Cette section présente l'intérêt des certifications industrielles, les raisons du choix de Java/JEE et le cahier des charges de la plateforme *NewroFactory*.

3.1.1 L'intérêt des certifications industrielles

Aujourd'hui, les clients grands comptes demandent des garanties concrètes sur le niveau technique des consultants qui rejoignent leurs équipes. Pour répondre à cette attente, Oxyl fait passer des certifications reconnues à tous ses nouveaux arrivants :

- **Oracle Certified Associate (OCA) et Oracle Certified Professional (OCP) Java SE** : validation de la maîtrise de Java (syntaxe, gestion de la mémoire JVM, concurrence, Streams, POO) – certificat OCA présenté en Annexe A (Figure A.1);
- **AWS Certified Cloud Practitioner** : validation des bases du Cloud, de la sécurité et des principaux services managés d'Amazon Web Services (Figure A.2).

Ces certifications permettent de valider objectivement un socle technique commun avant le départ en mission.

L'impact de la plateforme interne d'entraînement **MyNewroFactory (MNF)** est très net sur les temps de préparation :

- **Sans outil dédié** : la préparation demandait environ 3 mois pour l'OCA et près d'1 an pour l'OCP ;
- **Avec MNF** : la préparation de l'OCA prend **moins d'un mois** (durée divisée par trois) et celle de l'OCP environ 6 **mois** (durée divisée par deux).

Ce gain de temps permet aux consultants d'être disponibles plus rapidement pour les missions. La plateforme sert également à la formation continue et à la préparation des entretiens techniques de qualification client.

3.1.2 Le choix de Java et de la POO comme socle technique

Le choix de baser la formation initiale sur Java répond à plusieurs raisons techniques :

1. **La rigueur du typage et de la modélisation objet** : en tant que langage fortement typé, Java impose une structure claire (encapsulation, polymorphisme, interfaces, généricité) qui aide à bien modéliser des règles métier complexes ;
2. **La compréhension de l'exécution et de la mémoire** : appréhender le fonctionnement de la JVM (Garbage Collector, collections thread-safe, gestion des threads) est essentiel pour écrire du code performant et éviter les fuites de mémoire ;
3. **Une base pour comprendre les frameworks d'entreprise** : maîtriser Java pur permet de comprendre ce que font réellement des frameworks comme Spring Boot ou Hibernate, évitant ainsi d'utiliser des annotations sans savoir comment elles fonctionnent.

3.1.3 Limites de l'outil existant et besoins identifiés

L'ancien outil interne d'entraînement présentait quelques manques :

- **Peu de détails sur les résultats** : l'outil donnait seulement un score global en fin de série, sans détailler les réussites par chapitre ou par notion ;
- **Pas de vue globale pour les tuteurs** : les encadrants ne disposaient pas d'un tableau de bord pour suivre l'évolution d'une promotion ou repérer les notions posant problème ;
- **Manque d'explications interactives** : en cas d'erreur, l'apprenant avait peu d'éléments pour creuser le sujet en dehors de la réponse brute.

3.1.4 Cahier des charges et fonctionnalités de NewroFactory

Le cahier des charges de la nouvelle plateforme s'est concentré sur deux besoins : l'autonomie de l'apprenant et le suivi par les encadrants.

Fonctionnalités pour l'apprenant

Pour s'entraîner efficacement, le stagiaire dispose de :

- **Un tableau de bord détaillé** : visualisation des pourcentages de réussite par chapitre et du nombre de questions traitées pour identifier facilement les points faibles ;
- **Deux modes de passage** : un mode *Entraînement* avec correction immédiate et explications détaillées, et un mode *Examen blanc* chronométré en conditions réelles ;
- **Un bouton d'export JSON pour interroger un LLM** : en mode entraînement, un bouton permet de copier en un clic le contexte complet de la question (énoncé, code, propositions, explication) au format JSON (Figure 3.1). L'apprenant peut ainsi coller ces données dans ChatGPT ou Claude pour demander des éclaircissements ou des exemples supplémentaires sur la notion.

Fonctionnalités pour les encadrants

Pour l'administration et le suivi pédagogique :

- **Gestion des rôles** : séparation claire entre les profils stagiaires et les comptes administrateurs/tuteurs ;
- **Tableau de bord de promotion** : vue synthétique sur les résultats individuels et collectifs des stagiaires ;

OCA - Chapter 3 - JavaDataTypes_Q11/22 questions

Which of the following declarations does not compile?

☐ double num1, int num2 =0;

☒ int num1, num2 =0;

☒ int num1, num2;

☐ int num1 =0, num2 =0;

Sans contexte

Avec contexte

Finir la série

Suivant

FIGURE 3.1 – Interface de test en mode entraînement : correction immédiate et boutons d’export contextuel pour LLM

Série terminée !

Vous avez terminé l'entraînement sur le chapitre CH 3 Core Java APIs

0 / 1
0%

Insuffisant. Continuez à réviser ce chapitre pour vous améliorer !

Série complétée à 5%

Nouvelle série

Retour à l'accueil

FIGURE 3.2 – Écran de fin de série d’entraînement : score, taux de complétion et bilan par notion

- **Création et assignation de séries ciblées** : possibilité pour un tuteur de créer une série sur-mesure et de l'assigner à un stagiaire pour travailler une notion précise ;
- **Gestion du contenu pédagogique** : interface d'ajout et d'édition (CRUD) pour les chapitres, questions et propositions.

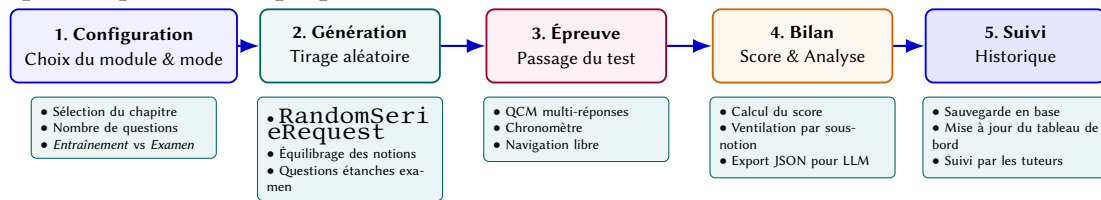


FIGURE 3.3 – Parcours utilisateur sur la plateforme NewroFactory

3.1.5 Objectifs de réalisation

L'objectif était de proposer une application complète et agréable à utiliser au quotidien :

- **Une interface soignée et réactive** : développée avec Angular, responsive et agréable visuellement ;
- **Une architecture backend robuste** : API REST en Spring Core découpée en couches étanches ;
- **La sécurité des données** : gestion propre des sessions et des rôles pour protéger les données d'évaluation.

3.2 Conception et architecture de la plateforme

Nous avons conçu la plateforme *NewroFactory* pour offrir aux stagiaires un environnement d'entraînement fidèle aux examens Oracle et AWS, tout en donnant aux encadrants une visibilité claire sur la progression de chacun.

3.2.1 Modélisation des données et arborescence des chapitres

Le premier travail a consisté à modéliser la hiérarchie des compétences sans pénaliser les performances de la base de données.

Organisation des chapitres par chemin matérialisé (*Materialized Path*)

Les programmes de certification sont hiérarchisés en modules, chapitres et sous-notions (par exemple : *Java Concurrency* → *Thread-Safe Collections* → *ConcurrentHashMap*). Pour stocker cette arborescence dans une base relationnelle sans utiliser de requêtes récursives lourdes, nous avons utilisé le patron de conception par **chemin matérialisé** (*Materialized Path*).

Dans l'entité `ChapterEntity`, chaque chapitre stocke la chaîne de ses parents dans le champ `parentPath` (ex. `" / 1 / 4 / "`) :

```

1 @Entity
2 @Table(name = "chapter", uniqueConstraints = @UniqueConstraint(columnNames = {
    "name", "parent_path" }))
  
```

```

3 @SoftDelete(columnName = "is_deleted")
4 public class ChapterEntity {
5
6     @Id
7     @GeneratedValue(strategy = GenerationType.IDENTITY)
8     private Long id;
9
10    @Column(name = "name", nullable = false)
11    private String name;
12
13    @Column(name = "parent_path")
14    private String parentPath;
15
16    @Formula("is_deleted")
17    private Boolean isDeleted;
18
19    // Getters, Setters et methodes metier
20 }

```

Listing 3.1 – Modélisation de ChapterEntity avec chemin matérialisé et Soft Delete

Ce choix simplifie grandement les requêtes SQL :

- Récupérer tous les sous-chapitres d'un nœud se fait avec une simple clause LIKE (parent_pathLIKE ' /1/4/%');
- Compter l'ensemble des questions d'un module et de ses sous-branches s'exécute en une seule requête indexée (countQuestionsByChapterIdsAndChildren);
- La cohérence de l'arbre est assurée par une contrainte d'unicité sur (name, parent_path) et une suppression logique (@SoftDelete).

La Figure 3.4 montre l'affichage dynamique de cette arborescence dans l'interface d'administration.

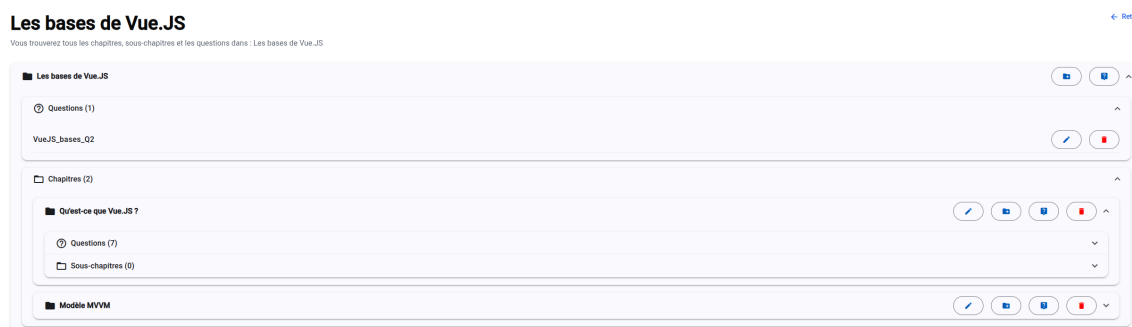


FIGURE 3.4 – Gestion de l'arborescence des chapitres utilisant le patron *Materialized Path*

Organisation des questions et des examens blancs

Les questions sont présentées sous forme de QCM conformes aux examens officiels. Afin que les examens blancs du vendredi conservent toute leur valeur de test, les questions sont séparées en deux catégories :

1. Les questions réservées aux examens blancs sont isolées dans un chapitre dédié;
2. Les sessions d'entraînement piochent aléatoirement dans le vivier d'entraînement général;
3. Les examens blancs utilisent exclusivement les questions réservées, avec le même nombre de questions et la même durée que l'épreuve officielle.

Modes d'évaluation : Entraînement et Examen

- **Mode Entraînement** : correction affichée dès la validation de la question, avec explications détaillées ;
- **Mode Examen** : pas d'indices en cours de test ; le score final est affiché à la fin sans le détail des réponses pour éviter le par-cœur lors des passages suivants.

Les Figures 3.5 et 3.6 illustrent la configuration d'une série par un utilisateur.

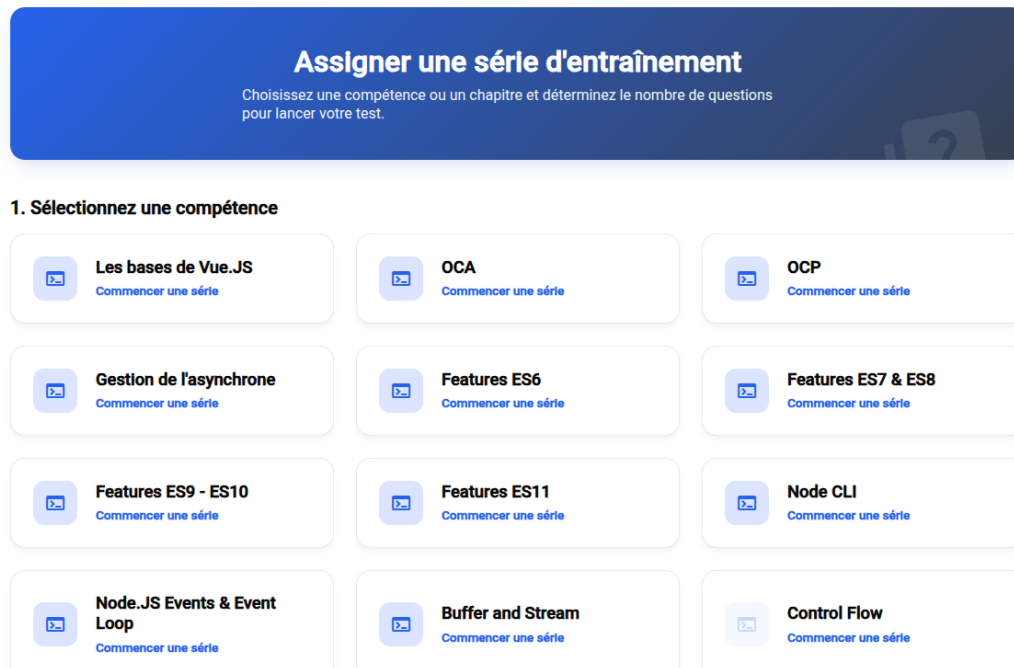


FIGURE 3.5 – Étape 1 : sélection du module cible (Vue.js, Java OCA/OCP, Node CLI, Spring)

3.2.2 Architecture logicielle backend (Spring)

Le backend est structuré en couches distinctes selon les principes de responsabilité unique (*Single Responsibility Principle*), d'inversion de dépendances et de *Clean Architecture* [7] (Figure 3.7).

Découpage en couches

- **Contrôleurs REST (`web.rest`)** : validation des données entrantes (Jakarta Validation) et renvoi de DTOs typés ;
- **Services métier (`service`)** : logique d'assemblage des séries (`RandomSerieRequest`), calcul des scores et gestion des utilisateurs ;
- **Persistence (`persistence`)** : accès aux données via Spring Data JPA et Hibernate.

Journalisation transverse avec Spring AOP

Pour éviter de polluer le code métier avec des logs, nous avons utilisé la programmation orientée aspect (*Spring AOP*) via le composant `LoggingAspect` (Listing 3.2).

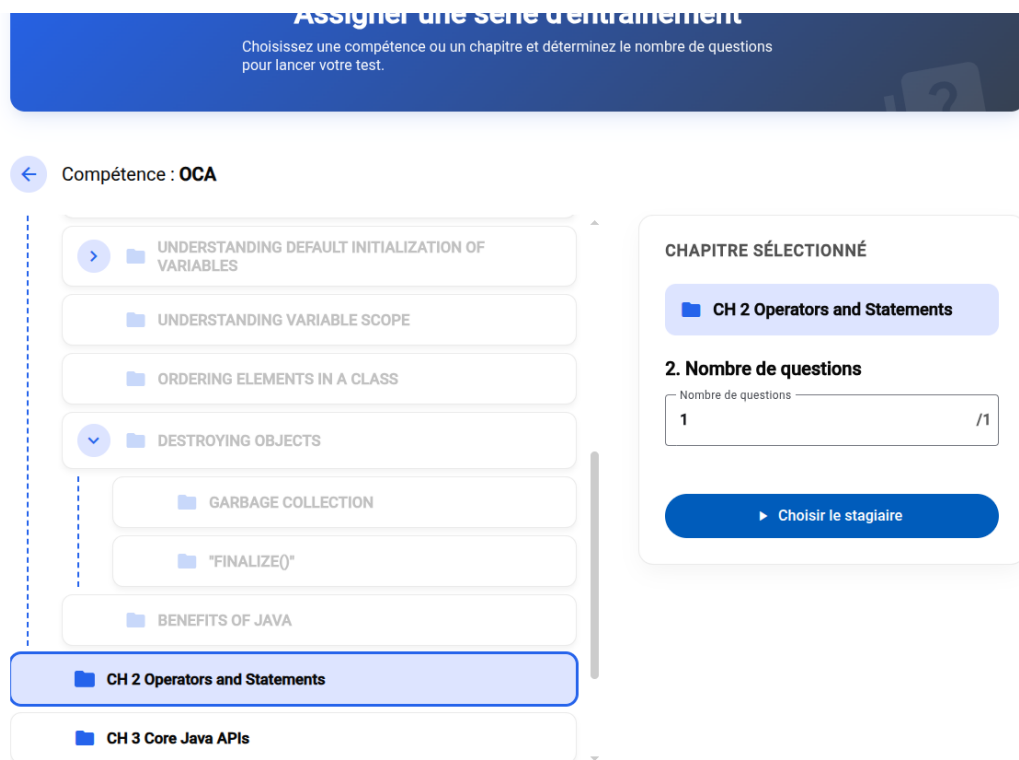


FIGURE 3.6 – Étape 2 : sélection du sous-chapitre et du nombre de questions

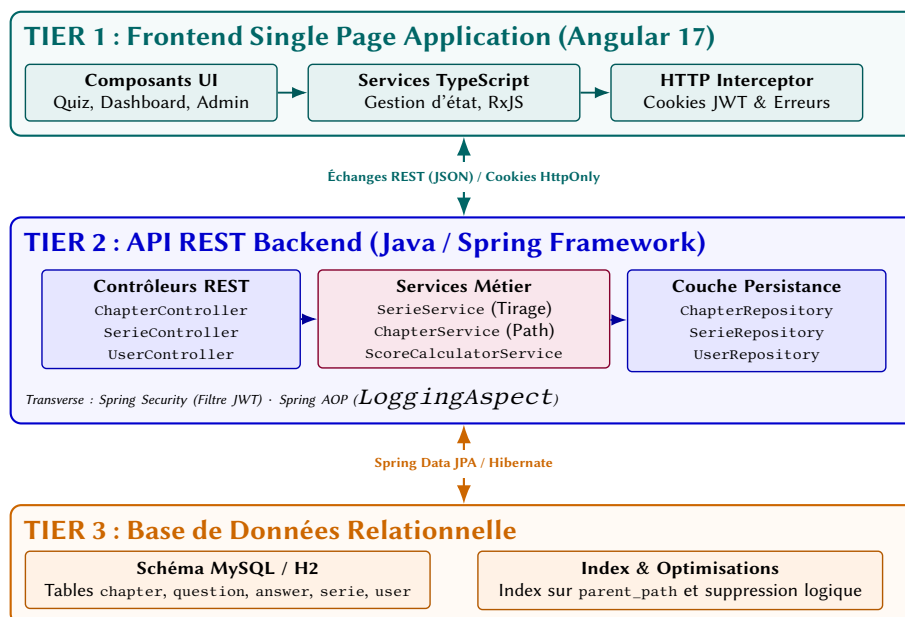


FIGURE 3.7 – Architecture 3-tiers de la plateforme NewroFactory

```

1 @Aspect
2 @Component
3 public class LoggingAspect {
4
5     @Pointcut("execution(public * com.oxyl.newrofactory.service.*(..)) || "
6             + "execution(public * com.oxyl.newrofactory.persistence.*(..))
7             || "
8             + "execution(public * com.oxyl.newrofactory.web.*(..))")
9     public void allLayersMethods() {}
10
11     @Before("allLayersMethods()")
12     public void logBefore(JoinPoint jp) {
13         getLogger(jp).info("-> Appel de : {}.{}",
14             jp.getTarget().getClass().getSimpleName(), jp.getSignature().
15             getName());
16     }
17
18     @AfterReturning(pointcut = "allLayersMethods()", returning = "result")
19     public void logAfterReturning(JoinPoint jp, Object result) {
20         getLogger(jp).info("<- Retour de : {} -> {}",
21             jp.getSignature().getName(), result);
22     }
23
24     @Pointcut("within(com.oxyl.newrofactory.persistence..*)")
25     public void persistenceMethods() {}
26
27     @AfterThrowing(pointcut = "persistenceMethods()", throwing = "exception")
28     public void translateDataAccessException(JoinPoint jp, DataAccessException
29     exception) {
30         Logger logger = getLogger(jp);
31         String className = jp.getTarget().getClass().getSimpleName();
32         String methodName = jp.getSignature().getName();
33         if (exception instanceof EmptyResultDataAccessException) {
34             logger.warn("! [FONCTIONNEL] Element non trouve dans {}.{}() : {}"
35                 ,
36                 className, methodName, exception.getMessage());
37         } else {
38             logger.error("!! [CRITIQUE] Erreur SQL/Base de donnees dans
39             {}.{}() : {}",
40                 className, methodName, exception.getMessage());
41         }
42     }
43 }

```

Listing 3.2 – Aspect transverse de logging et gestion des exceptions

Cet aspect enregistre automatiquement les entrées et sorties de méthodes et distingue les erreurs fonctionnelles simples (élément non trouvé, loggé en WARN) des pannes d'infrastructure (erreurs SQL, loggées en ERROR).

3.2.3 Sécurité et gestion des accès

Création des comptes et gestion des rôles

Pour sécuriser l'accès, l'inscription libre est désactivée. La création des comptes stagiaires est réservée aux administrateurs (ROLE_ADMIN), qui associent chaque stagiaire à sa promotion

(Figures 3.8 et 3.9). À sa première connexion, le stagiaire définit son mot de passe via l'écran update-password.

Gestion des utilisateurs [Ajouter un utilisateur](#) [Gestion des promotions](#) [Retour à l'accueil](#)

Gérer vos utilisateurs, vos stagiaires et leurs promotions

Rechercher par prénom Rechercher par nom Rechercher par email Chercher

Prénom	Nom de famille	Email	Stagiaire	Rôles	Actions
Abel	Bouille	abel.bouille@oxy.fr	✓		Utilisateur supprimé
Abelard	Grandjean	abelard.grandjean@oxy.fr	✓	User	✎ ✖
Abelin	Bourgignon	abelin.bourgignon@oxy.fr	✗	User	✎ ✖
Abraham	Delannoy	abraham.delannoy@oxy.fr	✗	User	✎ ✖
Achille	Charbonnier	achille.charbonnier@oxy.fr	✓	User	✎ ✖
Adam	Bouquet	adam.bouquet@oxy.fr	✓	User	✎ ✖
Adèle	Bellegarde	adele.bellegarde@oxy.fr	✓	User	✎ ✖
Adèle	Lavaud	adele.lavaud@oxy.fr	✓	User	✎ ✖
Adèle	Rouzet	adele.rouzet@oxy.fr	✓	User	✎ ✖
Admin	Admin	admin@oxy.fr	✗	Admin User	✎

FIGURE 3.8 – Gestion des utilisateurs et des promotions

Mise à jour

Rôles

USER

☒ Est-ce un stagiaire ?

Promotion*
Février 2011

Date de début
JJ/MM/AAAA

Date de fin
JJ/MM/AAAA

[Annuler](#) [Modifier](#)

FIGURE 3.9 – Modal d'édition d'un utilisateur (rôles et promotion)

Authentification JWT et configuration CORS

- Pour limiter les risques de failles XSS, le jeton JWT n'est pas stocké dans le `localStorage`, mais dans un cookie HTTP sécurisé (`HttpOnly`, `SameSite=Lax`);
- Pour faire communiquer le frontend Angular (port 4200 en local) et l'API Spring (port 8080), nous avons configuré les règles CORS dans `SecurityConfig` avec `setAllowCredentials(true)` et l'autorisation explicite des requêtes préliminaires (*preflight* OPTIONS).

3.2.4 Développement du frontend sous Angular

L'interface a été conçue sous Angular avec un accent mis sur la lisibilité et l'ergonomie.

Design System et styles CSS

J’ai pris en charge la conception de la feuille de style globale, du design system et de la mise en page générale :

- Définition d’une palette de variables CSS garantissant de bons contrastes (norme WCAG);
- Mise en page responsive adaptée aux écrans de bureau et portables;
- Création de composants visuels réutilisables : cartes de quiz (card-quiz), affichage de l’arbre des chapitres avec badges de score (chapter-tree-node) et modales de confirmation.

Parcours utilisateur

- **Tableau de bord stagiaire** : synthèse de la progression par module et historique des derniers examens (Figure 3.10);
- **Suivi des séries** : écran permettant de reprendre facilement un questionnaire entamé (Figure 3.11);
- **Interface d’administration** : réservée aux tuteurs pour gérer les promotions et consulter les résultats.

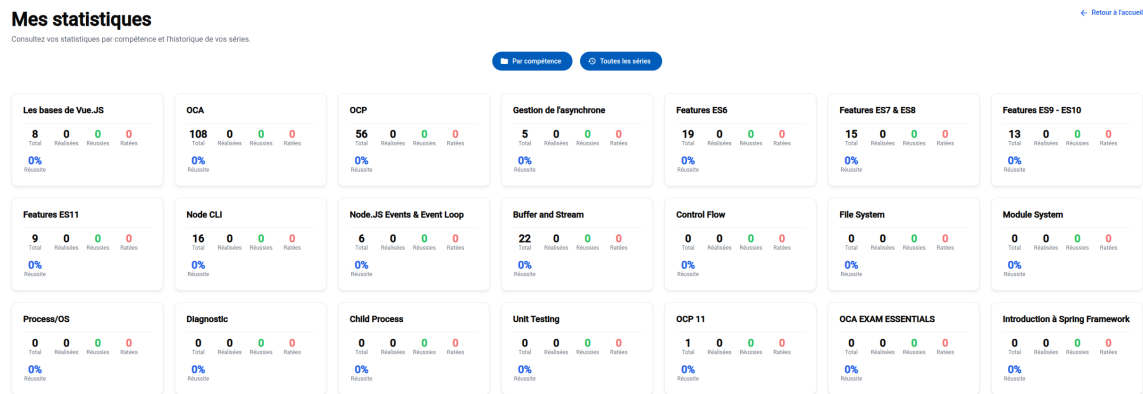


FIGURE 3.10 – Tableau de bord de suivi des compétences par chapitre

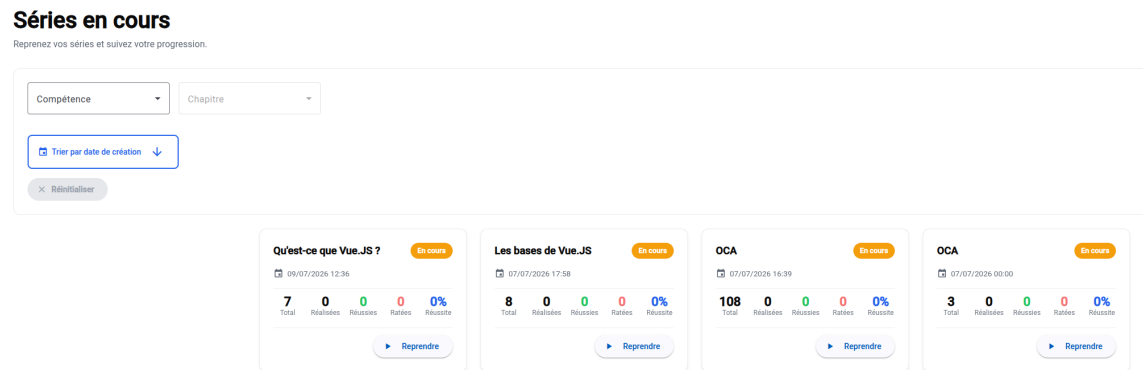


FIGURE 3.11 – Interface de suivi et de reprise des séries en cours

3.2.5 Organisation d'équipe et retours d'expérience

Au-delà de la technique, ce projet a constitué un exercice pratique de travail collaboratif en équipe de 4 stagiaires.

Organisation du développement

1. **Phase 1 (Sprints 1 à 8 – 2,5 mois)** : chaque stagiaire a d'abord développé son propre backend de bout en bout en Java (du CLI natif jusqu'à Spring Boot et Hibernate) pour bien assimiler les concepts ;
2. **Phase 2 (Sprint final)** : nous avons sélectionné l'un des backends comme base commune, puis nous avons développé ensemble le frontend Angular et finalisé les API.

Difficultés rencontrées et solutions

Travailler à quatre sur la même base de code a mis en évidence plusieurs points clés :

- **Gestion des branches Git** : au départ, des conflits de fusion survenaient par manque de synchronisations régulières (`git pull`). Nous avons rapidement mis en place une routine de synchronisation quotidienne systématique ;
- **Découpage des tickets** : nous avons constaté qu'il était indispensable de définir des tickets de petite taille et bien délimités pour éviter que deux développeurs ne modifient les mêmes fichiers simultanément ;
- **Synchronisation de la base de données** : avec des bases MySQL locales distinctes, les changements de schéma devaient être partagés via des scripts SQL, montrant tout l'intérêt d'outils de migration comme Flyway ou Liquibase ;
- **Déploiement Cloud** : l'application a été déployée sur Amazon Web Services à l'aide d'une instance virtuelle AWS EC2 pour les conteneurs et d'un bucket AWS S3 pour le stockage.

3.3 Justification des choix techniques

Les choix d'architecture ont été guidés par un équilibre entre apprentissage des fondamentaux et respect des délais.

3.3.1 Le choix de Spring Core et de la configuration explicite

Bien que Spring Boot permette de démarrer rapidement grâce à son auto-configuration, nous avons choisi d'utiliser explicitement **Spring Core** et ses modules :

- **Compréhension du conteneur IoC** : déclarer manuellement les Beans et leur cycle de vie permet de bien comprendre l'inversion de contrôle et l'injection de dépendances ;
- **Configuration des filtres et servlets** : mettre en place la chaîne de sécurité et les aspects de log montre comment fonctionne le moteur Web de Spring (`DispatcherServlet`) sans boîte noire ;
- **Architecture en couches** : structurer manuellement les flux entre contrôleurs, services et dépôts renforce les bonnes pratiques de séparation des responsabilités.

Ce choix permet à un ingénieur de comprendre les principes de base, ce qui rend la prise en main d'autres frameworks (Spring Boot, Quarkus, Micronaut) beaucoup plus rapide et intuitive.

3.3.2 Le choix d'Angular pour le frontend

Pour le client web, le choix d'**Angular** s'explique par :

- **La rigueur de TypeScript** : le typage statique permet de détecter les erreurs dès la compilation et s'aligne bien avec la rigueur du backend Java ;
- **La structure d'un framework d'entreprise** : Angular propose une organisation claire (composants, services, injection de dépendances) très utilisée sur les projets grands comptes chez Oxyl.

3.3.3 Monolithe modulaire vs Microservices

Nous avons opté pour un **monolithe modulaire** plutôt qu'une architecture en microservices :

- **Pragmatisme et délais** : les microservices auraient ajouté une complexité réseau et d'orchestration inutile pour l'échelle de ce projet ;
- **Focus sur le livrable** : l'objectif était d'obtenir une application fonctionnelle, bien testée et robuste dans le temps imparti.

3.3.4 Outils et bibliothèques complémentaires

- **Apache Maven** : gestion des dépendances et automatisation du build ;
- **Hibernate et JPA** : mapping objet-relationnel pour interagir simplement avec la base de données ;
- **Jakarta Validation** : validation automatique des DTOs en entrée d'API (@NotNull, @Size) ;
- **JWT (Java JWT)** : génération et vérification des jetons d'authentification pour sécuriser les échanges.

4 La standardisation du capital technique (Volet 2 - L'usine IA)

4.1 État de l'art : Industrialisation du SDLC et agents IA (SWE-bench)

L'utilisation des grands modèles de langage (*LLMs*) dans le cycle de développement logiciel (*SDLC*) ne se limite plus à l'autocomplétion de code dans l'éditeur. Elle évolue vers des agents autonomes capables de prendre en charge des séquences complètes de développement.

4.1.1 L'évolution du SDLC et la modification de code

Dans le cycle logiciel (analyse, conception, code, revue, tests, déploiement), la phase d'écriture et de modification du code reste la plus coûteuse en temps.

Les premiers benchmarks d'évaluation des LLM (comme HumanEval ou MBPP) mesuraient seulement la génération d'une fonction isolée. En pratique, le développement d'entreprise nécessite de comprendre une base de code complète, d'adapter des fichiers existants, de respecter les conventions du projet et de faire passer les tests d'intégration continue (CI/CD).

Dans l'usine logicielle d'Oxyl, le point d'entrée universel est le **ticket** (sur GitLab, GitHub ou Jira). L'usine traite ainsi trois types de besoins :

- **La correction de bugs (*bug fixing*)** : reproduction de l'erreur, localisation du problème et génération d'un patch de correction ;
- **L'ajout de fonctionnalités (*feature delivery*)** : création de nouveaux endpoints ou composants à partir de spécifications ;
- **Le refactoring et la dette technique** : mise à jour de dépendances, migration de syntaxe ou nettoyage de code mort.

4.1.2 Évaluation par SWE-bench et assistants en ligne de commande (CLI)

Pour évaluer les agents sur des cas réels, le benchmark **SWE-bench** [5] et les interfaces agentiques comme **SWE-agent** [9] se sont imposés comme des références. Ils testent la capacité d'un agent à résoudre de vrais tickets issus de projets open-source en modifiant le code jusqu'à validation de la suite de tests.

Pour intégrer ces capacités dans des pipelines automatisés, nous privilégions les outils utilisables en **ligne de commande (CLI)** :

- **Google Antigravity SDK** : pour orchestrer des agents spécialisés sur des tâches complexes ;
- **OpenAI Codex / CLI** [3] : pour l'édition ciblée de fichiers de code ;
- **Claude Code CLI** [1] : pour son raisonnement sur de larges fenêtres de contexte et son exécution fluide de commandes terminal.

4.1.3 Comparatif des solutions agentiques du marché

Le Tableau 4.1 compare l'approche retenue pour l'usine d'Oxyl avec les solutions existantes du marché.

TABLE 4.1 – Comparatif des solutions agentiques vs l'Usine IA Oxyl

Solution	Intégration CLI / CI-CD	Agnosticisme LLM (Anti lock-in)	Scope d'édition de code	Extensibilité des workflows
GitHub Copilot	Faible (IDE/Web)	Propriétaire (OpenAI)	Fichier / In-line	Limitée (Extensions)
Devin / SaaS	Web / Fermé	Propriétaire / Intégré	Projet complet	Configurable (SaaS)
Cursor / Claude Code	CLI / IDE local	Multi-modèles restreint	Projet complet	Scriptable
Usine IA Oxyl	Native CLI & CI-CD	Agnostique (Google/OpenAI/Claude)	Multi-fichiers complet	Totale (Patrons de conception & SDK)

Ce comparatif montre que l'usine d'Oxyl privilégie l'indépendance vis-à-vis des fournisseurs et le contrôle complet des étapes d'exécution via le terminal et la CI/CD.

4.2 Théorie de l'abstraction : Patrons de conception contre le lock-in

Chaque fournisseur de LLM (Anthropic, OpenAI, Google) utilise ses propres formats de requêtes, ses conventions d'outils (*tool calling*) et ses méthodes d'authentification.

Pour que l'usine reste pérenne et puisse changer de modèle sans réécrire l'orchestrateur, nous avons structuré le code autour de patrons de conception (*Design Patterns* [4]) classiques.

4.2.1 Choix et rôles des patrons de conception

- **L'Adapter (Adaptateur)** : c'est le cœur de l'abstraction. Il traduit les requêtes de l'usine (prompts, outils autorisés, contexte) vers le format attendu par le CLI ou l'API du modèle sélectionné, puis normalise la réponse.
- **La Strategy (Stratégie)** : l'application interagit avec une interface commune (*LLMProvider*). Le choix du moteur se fait dynamiquement à l'exécution en fonction de la tâche à accomplir.

- **Le Singleton / Registry pour l'instanciation** : plutôt que de recréer l'adaptateur à chaque appel, une instance unique est conservée en mémoire pendant la durée de vie du worker et accessible via un résolveur (`getLLMProvider()`).

4.2.2 Modélisation de la couche d'abstraction des LLMs

L'architecture d'abstraction repose sur une interface `LLMProvider`, un résolveur et des adaptateurs dédiés par modèle (Figure 4.1).

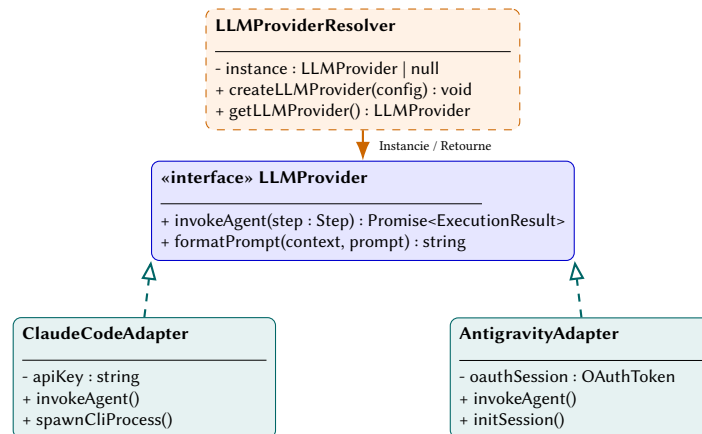


FIGURE 4.1 – Découplage des moteurs LLM par le patron Adapter

Grâce à cette interface, l'orchestrateur de l'usine ne dépend d'aucun SDK propriétaire.

4.2.3 Défis techniques d'intégration en conteneur

Faire tourner des agents en conteneur Docker sans interface graphique implique plusieurs ajustements :

1. **Modes d'exécution et authentification** :
 - **Claude Code CLI** : s'exécute directement en ligne de commande via `spawnSync` en passant le prompt sur `stdin` et la clé dans les variables d'environnement ;
 - **Google AntigraVity SDK** : nécessite une émulation de session pour tourner en mode conteneur sans interface graphique.
2. **Gestion des outils autorisés (*Tool Gating*)** : les étapes de lecture seule (revue, analyse) reçoivent uniquement les outils d'inspection (`Read`, `Glob`, `Grep`), tandis que l'outil de modification de fichier (`Edit`) est strictement désactivé pour éviter toute modification non voulue.
3. **Gestion des erreurs et des secrets** : en cas d'échec d'une étape, l'adaptateur prépare un résumé de l'erreur pour la tentative suivante tout en nettoyant les logs (`redactSecrets`) pour ne pas exposer de clés privées.

Ajouter un nouveau modèle (ex. un modèle local via Ollama ou vLLM) revient simplement à créer une nouvelle classe implémentant `LLMProvider`.

4.3 Gestion des contraintes opérationnelles : Coûts, sécurité et garde-fous

En production, l'utilisation d'agents IA nécessite de contrôler les coûts de tokens, de sécuriser l'accès au code et de s'assurer que les résultats produits sont fiables et reproductibles.

4.3.1 Maîtrise des coûts de tokens : Limites de réessais et disjoncteur

Pour éviter qu'un agent ne boucle indéfiniment en cas de problème et ne consomme des tokens inutilement, nous avons mis en place deux niveaux de contrôle (Figure 4.2) :

- **Plafond par étape (`max_retries`)** : chaque étape (analyse, génération, tests) autorise un nombre limité de tentatives (généralement 2 ou 3) ;
- **Disjoncteur global (`CircuitBreaker`)** : un compteur global d'essais (`globalAttempts`) est maintenu pour le ticket. S'il dépasse le seuil défini (`maxGlobalAttempts=25`), le traitement s'arrête immédiatement ;
- **Routage en cas d'erreur (`on_fail`)** : si une étape échoue, l'erreur est classifiée (`classifyFailure`) et le flux peut être redirigé vers une étape de diagnostic dédiée.

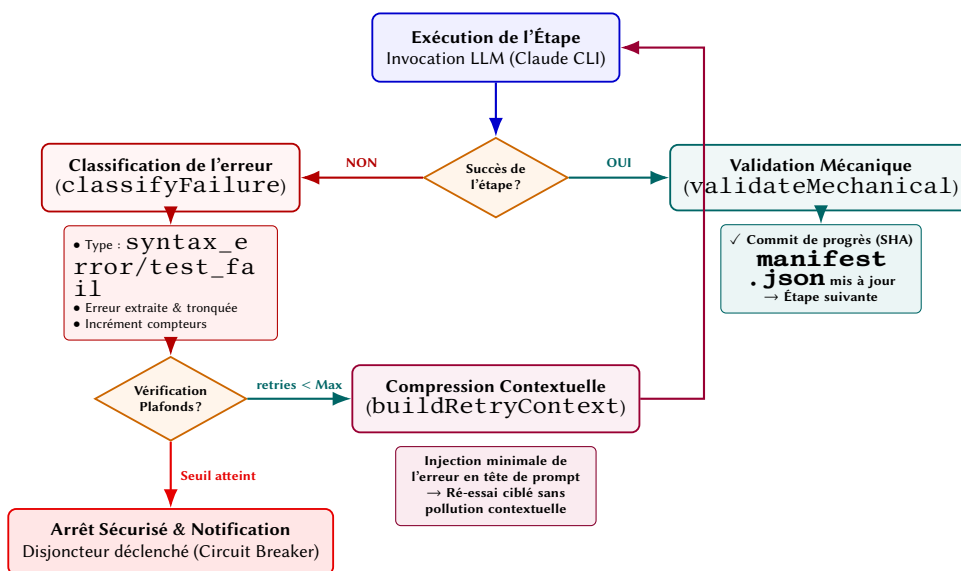


FIGURE 4.2 – Gestion des réessais et disjoncteur en cas d'erreur

4.3.2 Gestion du contexte et rôle du fichier Manifest

Les traces de logs et les sorties de tests peuvent rapidement saturer la fenêtre de contexte du modèle :

- **Troncature des logs** : le moteur tronque les messages longs en conservant uniquement le début et la fin (*head-and-tail truncation*) pour garder l'essentiel ;
- **Le fichier Manifest (`manifest.json`)** : l'état du traitement (étape en cours, numéro de tentative, extrait de l'erreur) est sauvegardé dans ce fichier structuré ;
- **Contexte de réessai minimal (`buildRetryContext`)** : en cas de réessai, seule l'erreur utile est réinjectée au début du prompt, évitant de renvoyer des milliers de lignes de logs inutiles.

4.3.3 Sécurité et protection des secrets

- **Respect des règles clients** : fonctionnement en local (*on-premise*) pour les clients stricts, ou utilisation d’accords d’entreprise garantissant la non-conservation des données (*zero data retention*) pour les services Cloud ;
- **Isolation en conteneurs Docker** : chaque agent tourne dans un conteneur éphémère avec des permissions limitées ;
- **Masquage des identifiants (`redactSecrets`)** : les clés d’API et tokens Git sont automatiquement masqués dans les logs et les rapports ;
- **Permissions limitées par étape** : interdiction d’utiliser l’outil d’écriture (`Edit`) lors des étapes de simple revue ou d’analyse.

4.3.4 Vérifications mécaniques et garde-fous

Comme les modèles peuvent parfois halluciner ou proposer du code incomplet, nous ne validons aucune étape sur la seule réponse de l’IA. Chaque résultat doit passer des vérifications automatiques déterministes.

La validation mécanique (`validateMechanical`)

À la fin de chaque étape, la fonction `validateMechanical(step)` vérifie :

- La présence physique des fichiers attendus sur le disque (ex. `.task/plan.md`) ;
- Des critères minimaux de longueur et de structure (expressions régulières) ;
- La présence d’un diff Git effectif pour les étapes d’implémentation (`git_diff_not_empty`).

Les scripts de garde-fous (*Mechanical Guards*)

Plusieurs scripts spécialisés contrôlent la conformité des modifications :

1. **Existence des fichiers sources (`check-source-files`)** : vérifie que les fichiers cités dans le plan existent réellement dans le dépôt ;
2. **Respect du périmètre (`check-scope`)** : s’assure que l’agent n’a modifié que les fichiers prévus dans son plan d’action ;
3. **Isolation des tests (`check-tests-scope`)** : pendant la rédaction des tests, l’agent ne peut pas modifier les fichiers sous `src/`, ce qui l’empêche d’altérer le code métier pour faire passer les tests ;
4. **Contrôle des dépendances (`check-new-deps`)** : bloque l’ajout imprévu de nouvelles bibliothèques dans `package.json` ;
5. **Contrôle des suppressions d’erreurs (`check-suppressions`)** : vérifie que l’agent n’a pas masqué des erreurs de typage avec des annotations comme `@ts-ignore` ou `eslint-disable`.

4.3.5 Observabilité et perspectives

L’agrégation de l’ensemble de ces garde-fous permet de transformer l’exécution des agents en un flux de travail prédictible et vérifiable.

Afin de consolider les métriques d'exécution, nous prévoyons l'intégration d'un composant d'observabilité fondé sur le patron de conception **Observer**. Ce module écouterait les événements émis par les superviseurs et les workers pour mesurer en temps réel :

- La consommation de tokens et les coûts associés par projet et par ticket ;
- La durée moyenne de traitement et le temps passé sur chaque étape ;
- Le taux de succès au premier essai (*First-Time Pass Rate*) et les erreurs les plus fréquentes arrêtées par les garde-fous.

5 La mise en œuvre de l'infrastructure agnostique

5.1 Architecture du moteur agnostique : Découplage de Git et des LLMs

Pour fonctionner dans des environnements variés, l'usine logicielle doit pouvoir s'interconnecter avec différents outils de tickets (GitLab, GitHub, Jira), différentes forges de code et différents modèles d'IA, tout en restant robuste et sécurisée.

L'ensemble de l'orchestrateur (Superviseur et Worker) est développé en **TypeScript** sous Node.js. Le typage statique strict et l'utilisation de types étiquetés (*Branded Types*) permettent d'éviter les erreurs d'inversion d'identifiants ou de tokens à la compilation.

5.1.1 Contrats d'interface : BacklogProvider et CodeBaseProvider

Pour découpler l'orchestration des outils externes, l'architecture sépare la gestion des tickets de la manipulation du code au travers de deux interfaces :

- **L'interface BacklogProvider (Gestion des tickets)** : côté superviseur (`src/supervisor/providers/backlog-provider.ts`), elle gère le cycle de vie des tickets :
 - `fetchReadyTickets()` : récupère les tickets ouverts portant le label d'éligibilité (ex. `factory::ready`);
 - `claimTicket(ticketId)` : réserve le ticket en posant un label de verrouillage (ex. `factory::claimed-by-<machine>`) pour éviter qu'un autre worker ne le prenne;
 - `addLabels, removeLabels` : met à jour les statuts du ticket;
 - `postComment(ticketId, body)` : publie les rapports et liens de suivi;
 - `getRelatedMergeRequests(ticketId)` : vérifie si une Merge Request existe déjà.
- **L'interface CodeBaseProvider (Gestion des branches et fusions)** : côté worker (`src/worker/providers/codebase-provider.ts`), elle unifie la création des demandes de fusion :
 - `handleMergeRequest(params)` : crée ou met à jour la *Merge Request* (GitLab) ou *Pull Request* (GitHub/Gitea) avec la branche source, la branche cible et la description du ticket.

Des adaptateurs sont déjà en place pour **GitLab** (`GitLabCodeBaseAdapter`) et **Gitea** (`GiteaCodeBaseAdapter`).

Grâce à cette séparation, un projet peut gérer ses tickets sur GitLab et son code sur GitHub ou Gitea sans changer une seule ligne du moteur de l'usine.

5.1.2 Types étiquetés (*Branded Types*) et résolution dynamique

Pour éviter de mélanger des identifiants ou des jetons de nature différente, nous utilisons des *Branded Types* dans `src/shared/providers/types.ts` (Listing 5.1).

```
1 declare const __brand: unique symbol;
2 export type Branded<T, B> = T & { [__brand]: B };
3
4 export type GitlabToken = Branded<string, 'GitlabToken'>;
5 export type GiteaToken = Branded<string, 'GiteaToken'>;
6
7 export type CodeBaseAccessInfo =
8   | { providerName: 'gitlab'; url: string; projectId: number; token:
      GitlabToken }
9   | { providerName: 'gitea'; url: string; owner: string; repo: string; token:
      GiteaToken };
```

Listing 5.1 – Types étiquetés et structures d'accès aux providers

Les résolveurs (`BacklogProviderResolver` et `CodeBaseProviderResolver`) appliquent le patron **Registry / Singleton** pour instancier une seule fois les adaptateurs nécessaires à partir du fichier de configuration du projet.

5.1.3 Cycle opérationnel : Isolation par *Git Worktrees*

Pour éviter de cloner tout le dépôt à chaque ticket, le superviseur utilise les **arborescences de travail Git** (*Git Worktrees*) :

1. **Boucle de surveillance et réservation (`pollCycle`)** : Le superviseur interroge périodiquement la forge, libère d'éventuels verrous expirés (`releaseExpiredClaims`), réserve un ticket disponible (`claimTicket()`) et lance le worker associé.
2. **Création de l'espace de travail isolé (`setup-worktree.ts`)** : Pour chaque tâche :
 - Une branche dédiée (ex. `feature/ticket-104`) et un répertoire isolé sous `.worktrees/<nom-branche>` sont créés via `createWorktree()` ;
 - Un dossier d'échange `.task/` est créé avec le fichier d'état `manifest.json` et automatiquement ajouté au `.gitignore` local ;
 - Les dépendances nécessaires sont installées proprement (`npmci`).
3. **Suivi des processus (*In-Flight Tracking*)** : Le superviseur enregistre les PIDs des workers en cours (`in-flight-store.ts`) et émet un signal de vie (`heartbeat`) régulier.

5.1.4 Exécution du pipeline et publication de la Merge Request

Une fois l'espace prêt, le Worker (`src/worker/worker.ts`) prend le relais :

- **Confinement de l'agent IA (`invokeAgent`)** : l'agent (Claude Code CLI ou Antigravity) est lancé en sous-processus avec des outils strictement limités (`--allowed-tools`). Il n'accède qu'aux fichiers locaux de son `worktree` ;
- **Nettoyage des logs (`redactSecrets`)** : tous les secrets et clés d'API sont automatiquement masqués dans les traces de logs ;
- **Publication déterministe de la MR (`create-mr.ts`)** : une fois les modifications validées par les tests et garde-fous mécaniques, l'IA ne touche plus à rien : c'est un script TypeScript

déterministe qui pousse la branche sur le dépôt distant et ouvre la Merge Request via l'adaptateur CodeBaseProvider.

5.1.5 Schéma d'architecture globale du moteur

La Figure 5.1 résume l'architecture globale du moteur agnostique et l'étanchéité entre la zone d'intervention de l'IA et les forges externes.

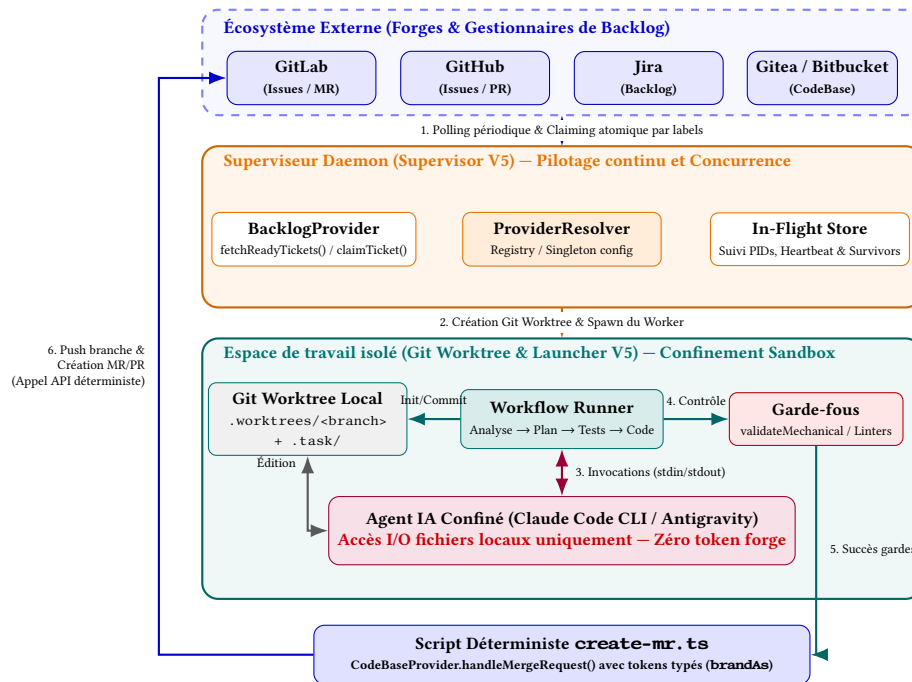


FIGURE 5.1 – Architecture globale du moteur agnostique : découplage des forges, isolation par Git Worktrees et étanchéité de l'agent IA

5.2 Stratégies de normalisation des forges et des agents

Pour unifier les échanges entre des outils hétérogènes, nous appliquons trois principes :

1. Un **modèle de données pivot** pour harmoniser les structures de tickets et de Merge Requests ;
2. Une **couche d'adaptation de CLI** pour normaliser les appels aux différents moteurs d'IA ;
3. Des **fichiers d'échange Markdown** sous `.task/` servant de contrat entre l'orchestrateur et l'agent IA.

5.2.1 Modèle pivot canonique : Unification des tickets et MRs

Chaque forge utilise ses propres formats (IDs de projet sous GitLab, couple owner/repo sous Gitea ou GitHub). Nous utilisons un modèle canonique unique couplé à des *Data Mappers* (Listing 5.2).

```
1 // 1. àModle canonique pivot pour les tickets
2 export type Ticket = {
3   id: number;
```

```

4  projectId: number | string;
5  title: string;
6  description: string;
7  labels: string[];
8  state: 'open' | 'closed';
9  priority: string;
10 };
11
12 // 2. àModle canonique pour les demandes de fusion (MR / PR)
13 export type MergeRequestResult = {
14   id: number;
15   iid: number;
16   url: string;
17   sourceBranch: string;
18   targetBranch: string;
19   state: 'opened' | 'closed' | 'merged';
20 };
21
22 // 3. Contrat d'adaptation Backlog
23 export interface BacklogProvider {
24   fetchReadyTickets(filter: TicketFilter, priorities: PriorityConfig): Promise
     <Ticket[] >;
25   claimTicket(ticketId: number): Promise<boolean >;
26   addLabels(ticketId: number, labels: string[]): Promise<void >;
27   removeLabels(ticketId: number, labels: string[]): Promise<void >;
28   postComment(ticketId: number, body: string): Promise<void >;
29   getRelatedMergeRequests(ticketId: number): Promise<RelatedMergeRequest[] >;
30 }

```

Listing 5.2 – Types pivots abstraits et interface BacklogProvider

Chaque adaptateur convertit les réponses des API distantes vers ces structures pivots, isolant complètement le reste de l'application des formats spécifiques de chaque fournisseur.

5.2.2 Utilisation des CLI agentiques autonomes

Nous avons choisi d'interagir avec les assistants en ligne de commande (**Claude Code CLI**, **Antigravity/Agy CLI**, **OpenAI Codex CLI**) plutôt qu'avec des appels d'API bruts :

- **Gestion intégrée du contexte** : les CLI gèrent déjà la compression de l'historique et les fenêtres de contexte;
- **Isolation des outils (*Tool Sandboxing*)** : le worker configure précisément les outils accessibles à chaque étape (Listing 5.3);
- **Sécurité financière** : chaque étape dispose d'un timeout et d'un plafond d'essais pour éviter les blocages.

```

1 export function getAllowedTools(step: Step): string {
2   const baseTools = ['Read', 'Write', 'Glob', 'Grep', 'Bash', 'Agent'];
3   // L'outil de 'dition ('Edit') est éédsactiv en analyse ou revue
4   if (!step.read_only) {
5     baseTools.push('Edit');
6   }
7   return baseTools.join(',');
8 }

```

Listing 5.3 – Génération dynamique des outils autorisés

5.2.3 Échange documentaire via les fichiers Markdown (`.task/* .md`)

Dans chaque *Git Worktree*, le dossier `.task/` centralise les artefacts au format Markdown qui font le lien entre l'orchestrateur et l'agent IA (Tableau 5.1).

TABLE 5.1 – Rôle des artefacts Markdown dans l'espace `.task/`

Fichier	Étape productrice	Contenu et rôle
<code>ticket.md</code>	Superviseur	Description du besoin, critères d'acceptation et branche cible.
<code>plan.md</code>	Agent (<code>plan-ticket</code>)	Plan d'implémentation : fichiers à modifier et stratégie de tests.
<code>staged-files.txt</code>	Garde-fous / Git	Liste des fichiers modifiés, utilisée par <code>check-scope</code> .
<code>feedback.md</code>	Worker (Validation)	Rapport d'erreurs réinjecté en entrée lors des réessais.
<code>review.md</code>	Agent (<code>review-code</code>)	Rapport d'audit de qualité et de non-régression.

Ce format Markdown est parfaitement compris par les modèles et reste lisible pour les développeurs lors de la relecture.

5.3 Validation et retours d'expérience sur cas réels

5.3.1 Expérimentations sur projets réels chez Oxyl

La chaîne de valeur a été testée et validée en plusieurs étapes :

1. **Test initial sur environnement Gitea** : développement d'une application de base de connaissances sous Angular sur une forge locale Gitea pour valider l'ensemble du cycle (polling, worktrees, création de PR) ;
2. **Projets réels en entreprise** :
 - **Projet Naskal** : plateforme e-commerce et de veille stratégique IT ;
 - **Projet BPI** : application interne d'analyse financière et de suivi de projets pour la Banque Publique d'Investissement.

Sur ces projets, l'usine a traité avec succès des tickets de correction de bugs et d'ajouts de composants.

5.3.2 Étude de cas : Résolution du Ticket #17

Le Ticket #17 (« *Fix element picker : escape key does not deactivate element picker* ») illustre le comportement réel du superviseur et du worker sous le workflow standard (Figure 5.2).

1. **Découverte et verrouillage** : le superviseur détecte le ticket #17 lors d'un cycle de polling, applique le label `claimed: *` pour bloquer les autres workers, puis lance le worker.
2. **Planification (plan)** : l'agent analyse le code en lecture seule et rédige le plan d'intervention dans `.task/plan.md`. La validation mécanique confirme qu'aucun fichier source n'a été modifié hors plan.

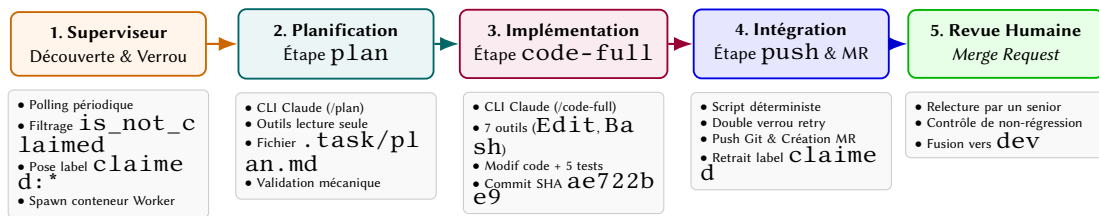


FIGURE 5.2 – Cycle de traitement d’un ticket par l’usine logicielle

- Développement (code-full)** : avec les outils d’édition activés, l’agent modifie les 4 fichiers nécessaires dans le worktree, écrit 5 tests unitaires (`Chat.elementPicker.spec.ts`) et crée un commit de sauvegarde (SHA `ae722be9`).
- Intégration et gestion d’une anomalie (push)** : lors du push vers le dépôt distant, une erreur d’authentification Git survient (identifiant manquant dans le conteneur). Le système de retry effectue deux tentatives avant de déclencher le disjoncteur (`maxRetries=2`) et d’arrêter le processus proprement sans bloquer la file d’attente.
- Revue humaine** : une fois la MR ouverte, un développeur senior effectue la relecture finale du code avant la fusion vers la branche principale.

Cette trace démontre que même face à une panne d’infrastructure réseau ou d’identifiant, les mécanismes de disjoncteur protègent l’usine de toute boucle infinie.

5.3.3 Le modèle Human-in-the-Loop et critères d’évaluation des LLMs

L’utilisation de l’usine logicielle modifie la place du développeur :

- Le développeur passe moins de temps sur le code de commodité (*boilerplate*) et se concentre sur la **revue de code (Code Review)** ;
- Il vérifie la pertinence des tests et la lisibilité du code (*Clean Code*) ;
- Il garde le contrôle final avant toute intégration en production.

Le Tableau 5.2 présente les indicateurs clés retenus pour évaluer et comparer les futurs moteurs LLM au sein de l’usine.

TABLE 5.2 – Critères d’évaluation et de comparaison des modèles d’IA

Axe d’évaluation	Indicateur (KPI)	Objectif mesuré
Qualité du code	Taux de réussite au 1 ^{er} tour	Pourcentage de tâches validant les tests dès la première tentative sans correction.
	Respect du plan	Concordance entre les fichiers modifiés et ceux annoncés dans le plan.
Autonomie	Vitesse de correction	Nombre moyen de cycles nécessaires pour corriger une erreur de compilation ou de test.
	Déclenchement du disjoncteur	Fréquence des échecs nécessitant une intervention manuelle.
Coûts & Efficience	Tokens consommés	Nombre moyen de tokens par ticket résolu.
	Temps de cycle	Durée entre la prise en charge du ticket et l’ouverture de la MR.
Revue humaine	Taux d’acceptation	Pourcentage de MRs acceptées sans retouches majeures.
	Lisibilité (<i>Clean Code</i>)	Clarté du nommage et respect de l’architecture existante.

Cette matrice permettra de conduire des campagnes d'évaluation objectives sur des jeux de tickets standardisés, assurant ainsi que l'usine logicielle sélectionne toujours le moteur le plus performant et le plus rentable pour chaque type de projet.

6 Analyse critique et impact organisationnel

6.1 Synergie entre formation humaine et usine logicielle

Chez Oxyl, la formation des collaborateurs (Volet 1) et l'automatisation par agents IA (Volet 2) ne sont pas deux démarches isolées. L'efficacité d'une usine logicielle dépend directement du niveau technique des ingénieurs qui cadrent les tâches et relisent le code produit.

6.1.1 Déplacement du travail et de la charge cognitive

En confiant à l'IA la rédaction du code répétitif (*boilerplate*, squelettes de classes, mappings simples), le développeur gagne du temps sur la saisie mécanique.

Ce temps est réinvesti dans des tâches à plus forte valeur ajoutée :

- **L'architecture et la modélisation** : découpage des modules, respect des contrats d'interfaces et scalabilité ;
- **La qualité du code (*Clean Code*)** : lisibilité, limitation du couplage et maintenabilité ;
- **L'arbitrage en cas de blocage** : si l'agent résout la tâche au premier essai, le gain est immédiat. En revanche, si l'IA s'égare après deux ou trois essais, l'ingénieur doit rapidement reprendre la main pour coder directement la solution plutôt que de perdre du temps à ajuster des prompts.

6.1.2 L'importance des fondamentaux théoriques pour relire le code de l'IA

L'utilisation de l'IA en entreprise fait apparaître un constat clair : **l'automatisation ne remplace pas l'expertise, elle la rend encore plus nécessaire.**

Lors d'une revue de code entre humains, la logique de l'auteur est généralement facile à suivre. Un modèle d'IA, quant à lui, peut proposer une solution qui semble fonctionner en apparence, mais qui introduit des effets de bord discrets, des fuites de ressources ou une complexité inutile. Relire ce type de code demande une grande rigueur technique.

C'est là que le socle de formation (notamment la préparation à l'OCP Java SE) prend tout son sens :

1. **Compréhension des mécanismes internes** : maîtriser la gestion de la mémoire, les collections thread-safe ou la gestion des exceptions est indispensable pour repérer les failles subtiles générées par l'IA ;

2. **Capacité de cadrage** : pour bien guider une IA par le prompt et les règles de validation, l'ingénieur doit lui-même connaître précisément les bonnes pratiques d'implémentation.

6.1.3 Transversalité des concepts : de Java à TypeScript

Bien que la plateforme d'évaluation (Volet 1) soit développée en Java et l'usine logicielle (Volet 2) en TypeScript, les principes de conception sont identiques :

- **La généricité et le typage strict** : les concepts de types paramétrés et de bornes génériques étudiés en Java facilitent la prise en main du typage avancé de TypeScript ;
- **Les patrons de conception (*Design Patterns*)** : les principes SOLID et les patrons d'architecture (Adapter, Strategy, Factory) s'appliquent de la même façon pour découpler les modules de l'usine.

6.1.4 Boucle d'apprentissage continu : Synergie MNF et Usine IA

À terme, Oxyl souhaite connecter directement la plateforme de formation **MyNewroFactory (MNF)** et l'**Usine Logicielle** :

1. L'usine analyse les revues de code et repère les notions techniques complexes sur lesquelles un projet a rencontré des difficultés (ex. gestion de la concurrence ou isolation transactionnelle) ;
2. Le système sélectionne automatiquement des questions d'entraînement ciblées dans la base de **MyNewroFactory** ;
3. Ces questions sont proposées au consultant lors de ses points de formation continue pour consolider ses acquis.

6.2 Impact économique, productivité et anti-lock-in

6.2.1 Retour sur investissement (ROI) et gains de temps

La rentabilité de l'usine logicielle repose sur la comparaison entre le coût d'inférence machine et le coût horaire d'un développeur.

En méthode traditionnelle, le traitement manuel d'un petit ticket de maintenance (création de DTO, squelette de service, écriture de tests de non-régression) demande 1 à 3 heures, soit un coût de 70 € à 250 € selon le profil.

Avec l'usine logicielle, le cycle complet consomme en moyenne 50 000 à 200 000 tokens, ce qui représente un coût direct d'environ 0,20 € à 1,50 € d'API par ticket. En ajoutant 10 à 15 minutes de revue par un ingénieur, le coût global est fortement réduit (Tableau 6.1).

Sur les projets pilotes, nous constatons :

- Une **réduction de 30 % à 50 % du temps de cycle** sur les tâches répétitives (génération de code CRUD, DTOs, tests simples) ;
- Une **résolution plus rapide des anomalies simples** dès leur signalement dans le backlog ;
- Un temps libéré pour que les équipes se concentrent sur la conception métier et l'architecture.

TABLE 6.1 – Comparatif économique : Traitement manuel vs Usine logicielle

Critère	Traitement manuel	Usine logicielle agentique
Temps de traitement	1 à 3 heures par micro-ticket	2 à 5 min d'exécution machine + 10 à 15 min de revue
Coût direct	70 € à 250 € (selon le TJM)	0,20 € à 1,50 € de jetons (+ temps de revue)
Rôle de l'ingénieur	Écriture intégrale du code et des tests	Cadrage du ticket et revue de code critique
Contrôle qualité	Revue par les pairs en fin de tâche	Vérifications automatiques systématiques (tests, linters)
Capacité de traitement	Limitée par le temps disponible	Traitement parallélisable via des conteneurs

6.2.2 Optimisation des coûts d'API et indépendance technologique

1. Routage dynamique des modèles : Tous les traitements ne nécessitent pas le modèle le plus puissant :

- Les modèles rapides et économiques (Claude Haiku, GPT-4o-mini ou modèles locaux) sont utilisés pour le tri des tickets, l'extraction de contexte et le formatage ;
- Les modèles avancés (Claude 3.5 Sonnet, GPT-4o) sont réservés aux étapes d'analyse de code multi-fichiers et de planification.

Ce découpage permet de réduire les dépenses d'API de plus de 60 % par rapport à l'utilisation systématique d'un modèle haut de gamme.

2. Protection contre le lock-in fournisseur : Grâce aux patrons *Adapter* et *Strategy*, l'usine peut changer de modèle ou basculer vers un modèle open-source local (ex. DeepSeek, Llama) en quelques minutes si les tarifs ou conditions d'un fournisseur évoluent.

6.2.3 Valorisation de la formation et réduction de la dette technique

- **Réduction des temps d'intercontrat :** passer la durée de préparation de l'OCA de 3 mois à moins d'un mois permet aux consultants d'intégrer des projets clients plus rapidement ;
- **Moins d'erreurs en production :** la bonne maîtrise des fondamentaux Java limite les fuites de mémoire et les anti-patterns, ce qui réduit le coût de maintenance des applications ;
- **Confiance des clients :** les certifications Oracle et AWS apportent une preuve objective du niveau technique des équipes lors des réponses aux appels d'offres.

6.3 Limites techniques, sécurité et impacts humains

6.3.1 Limites des modèles et importance de l'atomicité des tickets

L'utilisation pratique des LLMs met en évidence certains comportements à encadrer :

- **Hallucinations :** le modèle peut parfois inventer des méthodes ou des bibliothèques inexistantes dans le projet ;

- **Extrapolations non sollicitées** : si un cas limite n'est pas précisé dans le ticket, l'agent tente de deviner une règle métier, ce qui peut obliger à recommencer la tâche ;
- **Contournement des tests** : en cas d'erreur de test, l'agent modifie parfois le code de test ou supprime des vérifications pour faire passer le pipeline au vert au lieu de corriger le problème de fond.

Pour éviter ces dérives, la règle essentielle est l'**atomicité des tickets** : plus le périmètre d'un ticket est court, précis et bien délimité, plus l'agent produit un code juste dès le premier essai.

6.3.2 Sécurité et confidentialité des données

- **Filtrage des secrets** : les tokens, mots de passe et clés d'API sont automatiquement supprimés des logs et des requêtes envoyées aux modèles ;
- **Accords de non-conservation des données** : l'entreprise utilise des accès API professionnels garantissant que le code envoyé n'est pas conservé ni utilisé pour réentraîner les modèles publics ;
- **Option locale (*on-premise*)** : pour les clients aux règles de sécurité très strictes (banques, secteur public), l'usine peut fonctionner avec des modèles hébergés sur des serveurs privés.

6.3.3 Évolution du rôle du développeur et formation des juniors

L'automatisation change la manière dont les débutants apprennent le métier :

- **Apprentissage par la revue de code** : les juniors écrivent moins de code basique à la main, mais apprennent à analyser et critiquer le code généré par l'IA en binôme avec des développeurs seniors ;
- **Attention au biais de confiance** : la présentation soignée du code produit par l'IA peut inciter à le valider trop vite. Il reste indispensable de tester rigoureusement chaque modification ;
- **Fatigue liée à la relecture** : relire attentivement du code écrit par un tiers ou une machine demande un effort de concentration soutenu, d'où l'importance de garder des tickets de petite taille.

La Figure 6.1 résume cette évolution du temps de travail entre le développement traditionnel et l'approche assistée par l'usine logicielle.

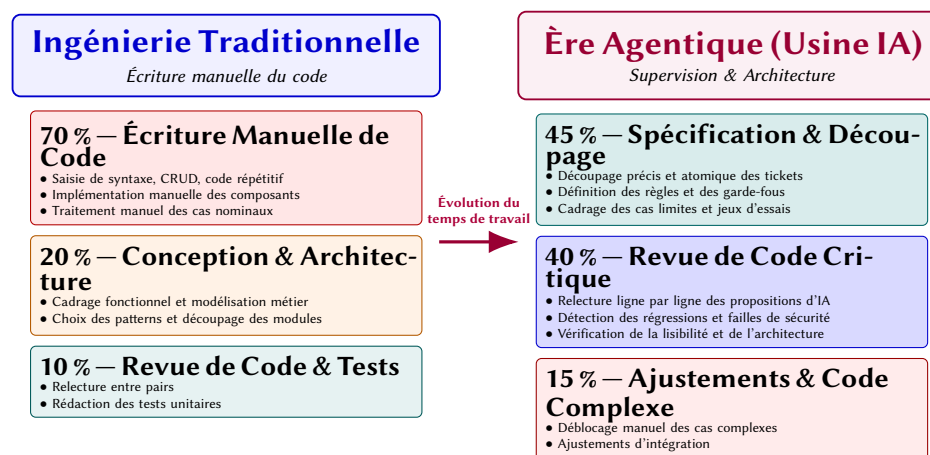


FIGURE 6.1 – Évolution de la répartition du temps de travail d'un ingénieur logiciel

7 Conclusion Générale

Bilan des réalisations

Ce projet de fin d'études a permis d'aborder la transformation des méthodes d'ingénierie chez Oxyl en combinant la formation aux fondamentaux techniques et l'automatisation logicielle par agents d'intelligence artificielle.

Les deux volets du stage ont abouti à des livrables fonctionnels :

- **La plateforme d'entraînement et d'évaluation (Volet 1)** : développée sous Spring Core et Angular, elle propose un environnement complet pour préparer les certifications du marché (Oracle Java OCA/OCP, AWS) avec un suivi statistique détaillé ;
- **L'infrastructure d'usine logicielle agnostique (Volet 2)** : développée en TypeScript, elle permet de traiter automatiquement des tickets de développement en s'interfaçant avec différentes forges (GitLab, Gitea) et différents modèles de langage (Claude, Antigravity, OpenAI), sans dépendance propriétaire.

Retombées et perspectives d'évolution

- **Pour la formation** : les concepts architecturaux mis en œuvre (arborescence par chemin matérialisé, authentification JWT sécurisée, design system moderne) fournissent une base d'inspiration pour faire évoluer les outils internes de l'entreprise.
- **Pour l'usine logicielle** : déployée et testée sur des projets réels (Naskal, BPI), l'usine traite déjà des tâches de développement courantes. Les prochaines étapes porteront sur l'amélioration de la transmission de contexte au modèle (via de la documentation structurée et du RAG) afin de réduire encore le nombre d'itérations nécessaires sur les tickets complexes.

Apprentissages et bilan personnel

Ce stage a été une expérience d'ingénierie très enrichissante :

- **Sur le plan technique** : j'ai pu approfondir l'architecture logicielle en Java/Spring et en TypeScript, expérimenter des patrons de conception pour assurer l'interopérabilité des systèmes et comprendre le fonctionnement concret des agents IA ;
- **Sur le plan méthodologique et humain** : le travail en équipe de 4 sur le premier volet m'a appris l'importance d'une bonne organisation (découpage fin des tickets, gestion rigoureuse des branches Git, communication régulière) et de la clarté dans la préparation des Merge Requests.

L'utilisation d'assistants IA ne diminue pas le rôle de l'ingénieur : elle le repositionne sur des activités d'architecture, de cadrage des besoins et de revue critique, où la rigueur et le recul technique restent indispensables.

Perspectives professionnelles

Fort des compétences consolidées durant cette formation d'ingénieur et ce stage, je souhaite poursuivre dans le domaine du génie logiciel et de l'automatisation par l'IA, en explorant des projets technologiques innovants tout en maintenant des liens étroits avec Oxyl.

Bibliographie / Webographie

- [1] Anthropic. The claude 3 model family : Opus, sonnet, haiku. <https://www.anthropic.com/news/claude-3-family>, 2024. 24
- [2] Jeanne Boyarsky and Scott Selikoff. *OCP Oracle Certified Professional Java SE 11 Developer Complete Study Guide : Exam 1Z0-819 and Upgrade Exam 1Z0-817*. Sybex / Wiley, 2020. 1
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Jared Kaplan, Harri Edwards, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. 24
- [4] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994. 24
- [5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Koustuv Kostic, and Karthik Narasimhan. Swe-bench : Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. 23
- [6] Robert C. Martin. *Clean Code : A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008. 6
- [7] Robert C. Martin. *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. 16
- [8] Craig Walls. *Spring in Action, Sixth Edition*. Manning Publications, 2022. 2
- [9] John Yang, Carlos E. Jimenez, Alexander Wettig, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent : Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv :2405.15793*, 2024. 23

Liste des illustrations

2.1	Parcours d'intégration et de formation chez Oxyl	7
2.2	Complémentarité entre compétences certifiées et usine logicielle agnostique . . .	9
3.1	Interface de test en mode entraînement : correction immédiate et boutons d'export contextuel pour LLM	13
3.2	Écran de fin de série d'entraînement : score, taux de complétion et bilan par notion	13
3.3	Parcours utilisateur sur la plateforme NewroFactory	14
3.4	Gestion de l'arborescence des chapitres utilisant le patron <i>Materialized Path</i> . . .	15
3.5	Étape 1 : sélection du module cible (Vue.js, Java OCA/OCP, Node CLI, Spring) . .	16
3.6	Étape 2 : sélection du sous-chapitre et du nombre de questions	17
3.7	Architecture 3-tiers de la plateforme NewroFactory	17
3.8	Gestion des utilisateurs et des promotions	19
3.9	Modal d'édition d'un utilisateur (rôles et promotion)	19
3.10	Tableau de bord de suivi des compétences par chapitre	20
3.11	Interface de suivi et de reprise des séries en cours	20
4.1	Découplage des moteurs LLM par le patron Adapter	25
4.2	Gestion des réessais et disjoncteur en cas d'erreur	26
5.1	Architecture globale du moteur agnostique : découplage des forges, isolation par Git Worktrees et étanchéité de l'agent IA	31
5.2	Cycle de traitement d'un ticket par l'usine logicielle	34
6.1	Évolution de la répartition du temps de travail d'un ingénieur logiciel	40
A.1	Certificat officiel Oracle Certified Associate (OCA) – Java SE 8 Programmer	55
A.2	Accréditation et badge officiel AWS Certified Cloud Practitioner	57

B.1	Diagramme de classes UML du domaine métier de la plateforme d'évaluation (Backend)	60
B.2	Architecture logicielle en couches et tissage des aspects transverses (Spring Core / AOP)	61
B.3	Architecture des composants réactifs et des flux de services (Angular Standalone)	63
B.4	Diagramme de classes UML des abstractions et adaptateurs de forges (Usine Logicielle)	65
B.5	Architecture du moteur de validation mécanique et boucle de réessai du Worker .	67
D.1	Page d'authentification et de connexion sécurisée à la plateforme NewroFactory .	73
D.2	Tableau de bord général de l'administrateur avec métriques consolidées et raccourcis de gouvernance	74
D.3	Catalogue et référentiel des compétences de formation et modules d'évaluation .	74
D.4	Interface de consultation et de filtrage dynamique de l'historique complet des séries	75

Liste des tableaux

4.1	Comparatif des solutions agentiques vs l'Usine IA Oxyl	24
5.1	Rôle des artefacts Markdown dans l'espace . task/	33
5.2	Critères d'évaluation et de comparaison des modèles d'IA	34
6.1	Comparatif économique : Traitement manuel vs Usine logicielle	39

Listings

3.1	Modélisation de ChapterEntity avec chemin matérialisé et Soft Delete	14
3.2	Aspect transverse de logging et gestion des exceptions	18
5.1	Types étiquetés et structures d'accès aux providers	30
5.2	Types pivots abstraits et interface BacklogProvider	31
5.3	Génération dynamique des outils autorisés	32
C.1	Extrait des traces d'exécution de l'usine logicielle sur le Ticket #17	69

Glossaire

CLI *Command-Line Interface*. Interface utilisateur textuelle manipulée et exécutée en ligne de commande 51

DTO / DAO *Data Transfer Object / Data Access Object*. Patrons de conception dédiés respectivement au transport de données structurées et à l'abstraction de la couche de persistance 51

ESN Entreprise de Services du Numérique. Société de conseil et d'ingénierie délivrant des prestations logicielles en régie ou au forfait 51

LLM *Large Language Model*. Grand modèle de traitement automatique du langage fondé sur des réseaux de neurones profonds (*Transformers*) 51

MR / PR *Merge Request / Pull Request*. Demande formelle de fusion d'une branche de fonctionnalités vers la branche principale, soumise à une revue de code collaborative 51

OCA *Oracle Certified Associate*. Certification industrielle validant l'acquisition des fondamentaux de la programmation Java 51

OCP *Oracle Certified Professional*. Certification professionnelle de référence sanctionnant la maîtrise avancée de la plateforme Java SE 51

RAG *Retrieval-Augmented Generation*. Technique combinant un mécanisme de recherche documentaire et un modèle génératif pour contextualiser précisément les réponses 51

REST *Representational State Transfer*. Style architectural standardisé fondé sur les protocoles et verbes HTTP pour la communication entre services web 51

ROI *Return on Investment* (Retour sur Investissement). Indicateur financier mesurant la rentabilité et les gains économiques générés par un investissement 51

SDLC *Software Development Life Cycle*. Cycle de vie global du développement logiciel, de l'analyse du besoin au maintien en condition opérationnelle 51

SOLID Ensemble de cinq principes de conception orientée objet (*Single responsibility, Open-closed, Liskov substitution, Interface segregation, Dependency inversion*) favorisant la maintenabilité et l'extensibilité du code 51

TJM Taux Journalier Moyen. Métrique tarifaire de référence représentant le coût de facturation journalier d'un ingénieur consultant 51

Vendor Lock-in Enfermement ou dépendance propriétaire. Situation de dépendance technologique où un système ne peut changer de fournisseur sans surcoûts majeurs 51

Annexes

A Attestations des certifications obtenues

Cette annexe regroupe les justificatifs et accréditations officielles validant l'obtention des certifications industrielles durant le stage au sein du Groupe Oxyl.

A.1 Oracle Certified Associate (OCA) – Java SE 8 Programmer

La Figure A.1 reproduit l'attestation officielle délivrée par Oracle University certifiant la réussite de l'examen *Oracle Certified Associate, Java SE 8 Programmer*.



FIGURE A.1 – Certificat officiel Oracle Certified Associate (OCA) – Java SE 8 Programmer

A.2 AWS Certified Cloud Practitioner

La Figure A.2 présente le badge et l'accréditation officielle délivrée par Amazon Web Services Training and Certification validant le titre *AWS Certified Cloud Practitioner*.



FIGURE A.2 – Accréditation et badge officiel AWS Certified Cloud Practitioner

B Diagrammes de classe et Schémas d'architecture logicielle

Cette annexe rassemble les diagrammes de classes UML et les schémas d'architecture détaillant la conception structurelle des différents systèmes développés au cours du projet de fin d'études, extraits directement de l'analyse des dépôts de code sources réels (`newro-factory-back-avril`, `newro-factory-front-avril` et `usine-logicielle`).

B.1 Plateforme d'Évaluation : Modèle du Domaine Métier (Backend Spring)

Le diagramme de la Figure B.1 présente les classes du domaine métier de l'application backend. Il met en lumière l'organisation arborescente des connaissances via le patron de chemin matérialisé (`parentPath`), l'étanchéité des séries d'entraînement et d'examen (`Serie`, `SerieQuestion`, `SerieReponse`), ainsi que la gestion de l'historique et des utilisateurs.

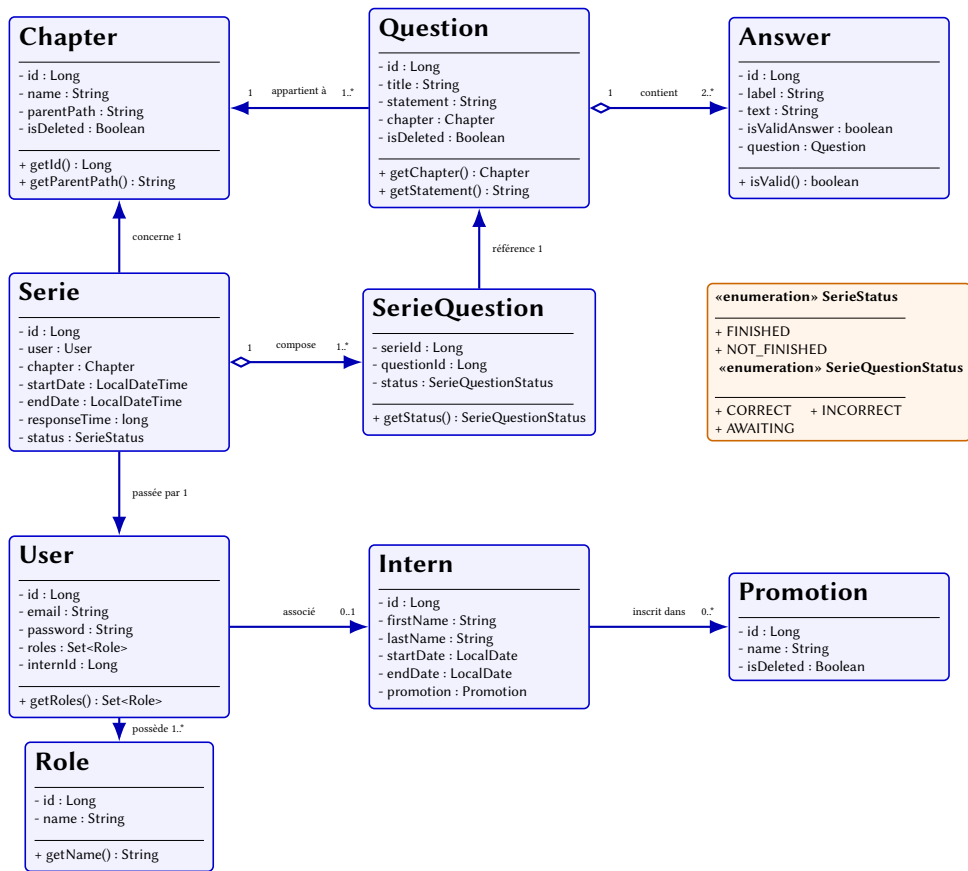


FIGURE B.1 – Diagramme de classes UML du domaine métier de la plateforme d'évaluation (Backend)

B.2 Plateforme d'Évaluation : Architecture Logicielle en Couches et Aspects (Spring)

La Figure B.2 détaille le découpage en couches de l'application backend Java, illustrant le flux d'exécution type d'une requête REST (du contrôleur jusqu'à la base de données via DAOs et Repositories), ainsi que l'interception non-intrusive assurée par Spring AOP pour la journalisation, la sécurité et la métrologie de performance.

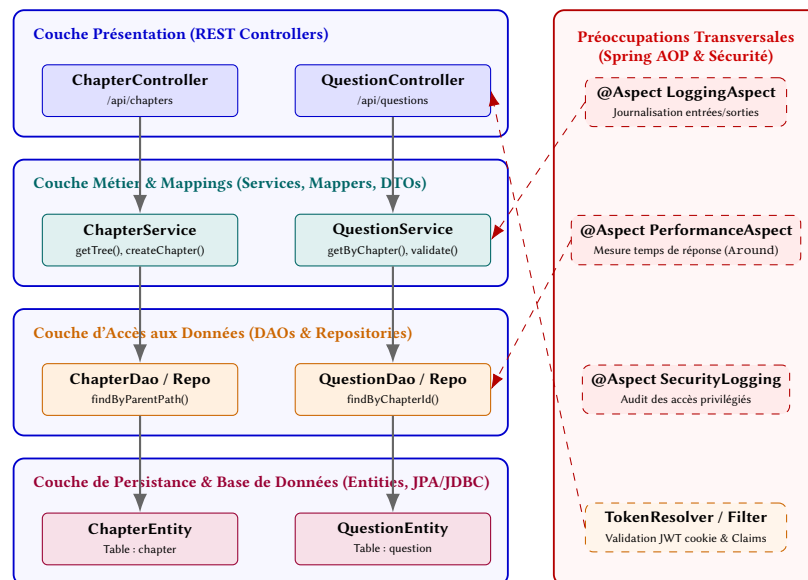


FIGURE B.2 – Architecture logicielle en couches et tissage des aspects transverses (Spring Core / AOP)

B.3 Plateforme d'Évaluation : Architecture Frontend Réactive (Angular)

La Figure B.3 présente la structure de l'application cliente Angular du projet newro-factory-front-avril. Elle met en évidence l'architecture modulaire séparant les composants conteneurs dynamiques (*Smart Components*), les composants de présentation réutilisables (*Dumb Components*), ainsi que les services d'infrastructure et d'accès aux API distantes.

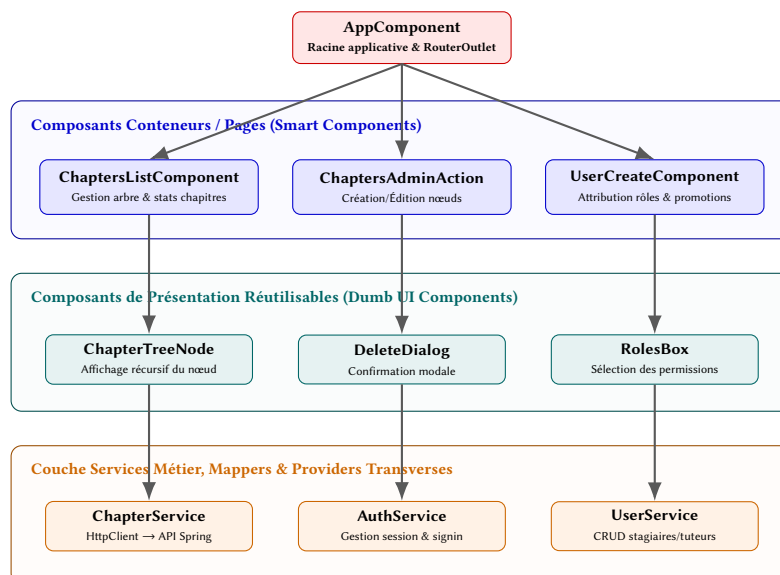


FIGURE B.3 – Architecture des composants réactifs et des flux de services (Angular Standalone)

B.4 Usine Logicielle : Patrons de Conception et Abstractions des Fournisseurs

La Figure B.4 expose les diagrammes de classes UML régissant le découplage technologique de l'usine logicielle du projet `usine-logicielle`. On y retrouve l'application combinée des patrons *Adapter*, *Factory/Resolver* et *Branded Types* pour garantir l'interchangeabilité des forges logicielles (`BacklogProvider`, `CodeBaseProvider`) sans couplage fort avec les bibliothèques externes.

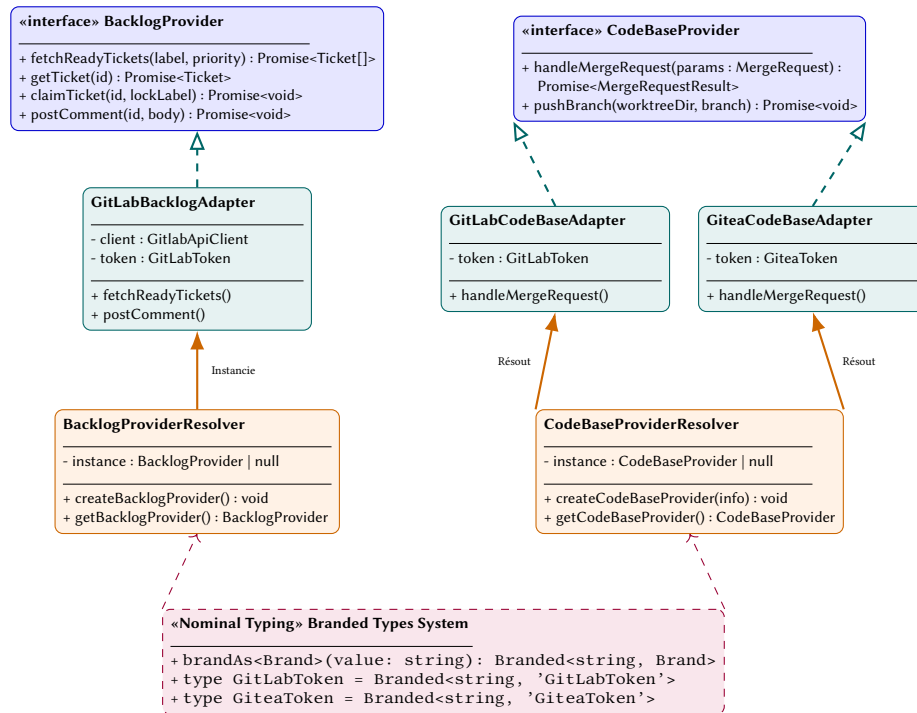


FIGURE B.4 – Diagramme de classes UML des abstractions et adaptateurs de forges (Usine Logicielle)

B.5 Usine Logicielle : Moteurs de Validation Mécanique et Cycle du Worker

La Figure B.5 schématise l'organisation interne du moteur d'exécution (**Worker**) et de ses garde-fous mécaniques, garantissant l'étanchéité contextuelle et la résilience aux pannes via la persistance d'états dans le fichier `manifest.json`.

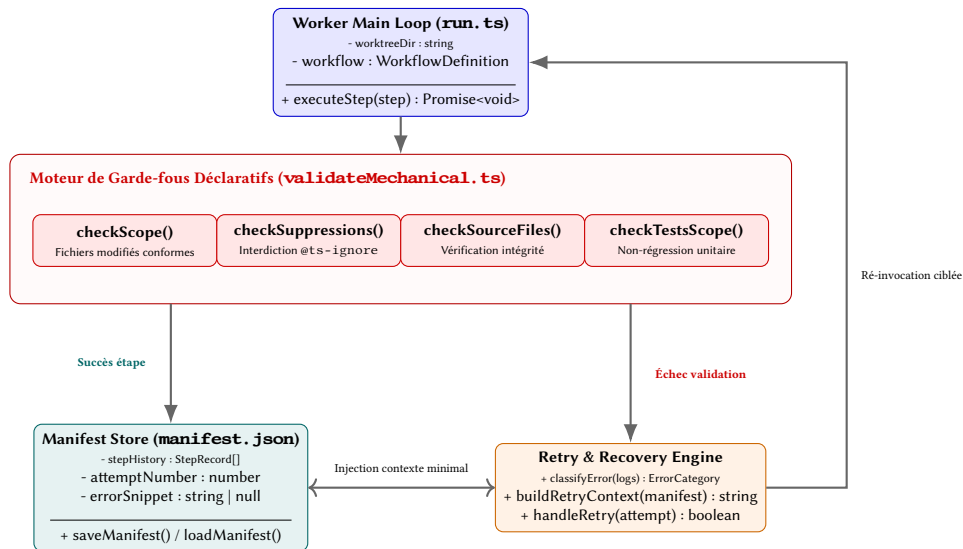


FIGURE B.5 – Architecture du moteur de validation mécanique et boucle de réessai du Worker

C Exemples de traces d'exécution de l'usine logicielle

Cette annexe présente un relevé exhaustif de journaux d'audit (*execution logs*) générés en environnement conteneurisé lors du traitement automatisé d'un ticket d'ingénierie (Ticket #17 : « *Fix element picker : escape key does not deactivate element picker* »).

Les événements sont émis au format JSON Lines structuré, intégrant l'horodatage ISO-8601 (ts), le niveau de sévérité (level), le composant émetteur (component : supervisor ou worker), l'identifiant d'exécution unique (runId), ainsi que les métriques et métadonnées d'étape.

C.1 Journal brut d'exécution (Format JSON Lines)

Le Listing C.1 reproduit la séquence complète des logs, depuis la finalisation du plan d'action jusqu'à la détection de défaillance d'authentification lors de l'étape finale d'intégration Git.

```
1 usine-usine | {"ts":"2026-08-27T15:58:00.937Z","level":"INFO","component":"worker","msg":"Claude execution completed","runId":"1017-17-20260827T155453Z","stepId":"plan","exitCode":0,"outputLength":409,"output":"Ticket #17's implementation plan is ready in .task/plan.md. It covers: adding an ELEMENT_PICKER_CANCEL endpoint, a new cancel handler in both the content-script and background element picker controllers, an Escape-key listener in Chat.vue (sidebar) gated on picker-active state with proper mount/unmount cleanup, and matching test cases in Chat.spec.ts. No source files were modified -- just the plan.\n","stderrLength":0,"stderrlog":""}
2 usine-usine | {"ts":"2026-08-27T15:58:00.938Z","level":"DEBUG","component":"worker","msg":"Global attempt incremented","runId":"1017-17-20260827T155453Z","total":2}
3 usine-usine | {"ts":"2026-08-27T15:58:00.939Z","level":"DEBUG","component":"worker","msg":"Mechanical validation passed","runId":"1017-17-20260827T155453Z","stepId":"plan"}
4 usine-usine | {"ts":"2026-08-27T15:58:00.939Z","level":"INFO","component":"worker","msg":"Step execution completed","runId":"1017-17-20260827T155453Z","stepId":"plan","success":true,"exitCode":0}
5 usine-usine | {"ts":"2026-08-27T15:58:01.159Z","level":"WARN","component":"worker","msg":"Commit failed","runId":"1017-17-20260827T155453Z","stepId":"plan","error":"Command failed: git commit -m usine: plan -- success"}
6 usine-usine | {"ts":"2026-08-27T15:58:01.161Z","level":"INFO","component":"worker","msg":"Step notification","runId":"1017-17-20260827T155453Z","stepId":"plan","status":"success","durationMs":65220,"nextStepId":"code-full"}
7 usine-usine | {"ts":"2026-08-27T15:58:01.161Z","level":"DEBUG","component":"worker","msg":"Execution log appended","runId":"1017-17-20260827T155453Z","stepId":"plan","totalEntries":2}
8 usine-usine | {"ts":"2026-08-27T15:58:01.170Z","level":"DEBUG","component":"worker","msg":"Manifest saved","runId":"1017-17-20260827T155453Z","path":"/repo/worktrees/17/.task/manifest.json","currentStep":"code-full"}
9 usine-usine | {"ts":"2026-08-27T15:58:01.170Z","level":"INFO","component":"worker","msg":"=== Step starting","runId":"1017-17-20260827T155453Z","stepId":"code-full","index":2,"dispatchReason":"initial"}
10 usine-usine | {"ts":"2026-08-27T15:58:01.174Z","level":"DEBUG","component":"worker","msg":"Step execution starting","runId":"1017-17-20260827T155453Z","stepId":"code-full","type":"prompt"}
11 usine-usine | {"ts":"2026-08-27T15:58:01.174Z","level":"DEBUG","component":"worker","msg":"Tools allocated for step","runId":"1017-17-20260827T155453Z","stepId":"code-full","tools":["Read","Write","Glob","Grep","Bash","Edit","Task"]}
12 usine-usine | {"ts":"2026-08-27T15:58:01.175Z","level":"INFO","component":"worker","msg":"Executing prompt step","runId":"1017-17-20260827T155453Z","stepId":"code-full","command":"/code-full","tools":7}
13 usine-usine | {"ts":"2026-08-27T15:58:01.177Z","level":"DEBUG","component":"worker","msg":"Spawning claude CLI (prompt sent via stdin)","runId":"1017-17-20260827T155453Z","binary":"claude","args":["--print","--output-format","text","--verbose","--session-id","777a71ca-b8ce-4507-9876-669a2d2aee47","--allowed-tools","Read,Write,Glob,Grep,Bash,Edit,Task"]}
14 usine-usine | {"ts":"2026-08-27T15:58:06.263Z","level":"DEBUG","component":"supervisor","msg":"Poll cycle #20","activeWorkers":1,"inFlightTickets":1}
15 usine-usine | {"ts":"2026-08-27T15:58:26.267Z","level":"DEBUG","component":"supervisor","msg":"Poll cycle #21","activeWorkers":1,"inFlightTickets":1}
16 usine-usine | {"ts":"2026-08-27T15:58:46.275Z","level":"DEBUG","component":"supervisor","msg":"Poll cycle #22","activeWorkers":1,"inFlightTickets":1}
17 usine-usine | {"ts":"2026-08-27T15:59:06.272Z","level":"DEBUG","component":"supervisor","msg":"Poll cycle #23","activeWorkers":1,"inFlightTickets":1}
18 usine-usine | {"ts":"2026-08-27T15:59:26.278Z","level":"DEBUG","component":"supervisor","msg":"Poll cycle #24","activeWorkers":1,"inFlightTickets":1}
19 usine-usine | {"ts":"2026-08-27T15:59:27.354Z","level":"DEBUG","component":"supervisor","msg":"found workflow: usine-PierreV:full for ticket id=17","id":17}
20 usine-usine | {"ts":"2026-08-27T15:59:27.359Z","level":"DEBUG","component":"supervisor","msg":"[ELIGIBILITY] #17 rejete par \"is_not_claimed\": label claimed: \"present\",\"iid\":17,\"rule\":\"is_not_claimed\"}
```

```

21 usine-usine | {"ts":"2026-08-27T15:59:46.288Z","level":"DEBUG","component":"supervisor","msg":"Poll cycle #25","
    activeWorkers":1,"inFlightTickets":1}
22 usine-usine | {"ts":"2026-08-27T15:59:47.172Z","level":"DEBUG","component":"supervisor","msg":"found workflow: usine-
    PierreV::full for ticket id=17","id":17}
23 usine-usine | {"ts":"2026-08-27T15:59:47.173Z","level":"DEBUG","component":"supervisor","msg":"[ELIGIBILITY] #17 rejete
    par \"is_not_claimed\": label claimed:* present","iid":17,"rule":"is_not_claimed"}
24 usine-usine | {"ts":"2026-08-27T16:00:06.296Z","level":"DEBUG","component":"supervisor","msg":"Poll cycle #26","
    activeWorkers":1,"inFlightTickets":1}
25 usine-usine | {"ts":"2026-08-27T16:00:07.026Z","level":"DEBUG","component":"supervisor","msg":"found workflow: usine-
    PierreV::full for ticket id=17","id":17}
26 usine-usine | {"ts":"2026-08-27T16:00:07.026Z","level":"DEBUG","component":"supervisor","msg":"[ELIGIBILITY] #17 rejete
    par \"is_not_claimed\": label claimed:* present","iid":17,"rule":"is_not_claimed"}
27 usine-usine | {"ts":"2026-08-27T16:00:26.307Z","level":"DEBUG","component":"supervisor","msg":"Poll cycle #27","
    activeWorkers":1,"inFlightTickets":1}
28 usine-usine | {"ts":"2026-08-27T16:00:27.265Z","level":"DEBUG","component":"supervisor","msg":"found workflow: usine-
    PierreV::full for ticket id=17","id":17}
29 usine-usine | {"ts":"2026-08-27T16:00:27.265Z","level":"DEBUG","component":"supervisor","msg":"[ELIGIBILITY] #17 rejete
    par \"is_not_claimed\": label claimed:* present","iid":17,"rule":"is_not_claimed"}
30 usine-usine | {"ts":"2026-08-27T16:00:46.315Z","level":"DEBUG","component":"supervisor","msg":"Poll cycle #28","
    activeWorkers":1,"inFlightTickets":1}
31 usine-usine | {"ts":"2026-08-27T16:00:47.417Z","level":"DEBUG","component":"supervisor","msg":"found workflow: usine-
    PierreV::full for ticket id=17","id":17}
32 usine-usine | {"ts":"2026-08-27T16:00:47.418Z","level":"DEBUG","component":"supervisor","msg":"[ELIGIBILITY] #17 rejete
    par \"is_not_claimed\": label claimed:* present","iid":17,"rule":"is_not_claimed"}
33 usine-usine | {"ts":"2026-08-27T16:01:06.315Z","level":"DEBUG","component":"supervisor","msg":"Poll cycle #29","
    activeWorkers":1,"inFlightTickets":1}
34 usine-usine | {"ts":"2026-08-27T16:01:07.107Z","level":"DEBUG","component":"supervisor","msg":"found workflow: usine-
    PierreV::full for ticket id=17","id":17}
35 usine-usine | {"ts":"2026-08-27T16:01:07.108Z","level":"DEBUG","component":"supervisor","msg":"[ELIGIBILITY] #17 rejete
    par \"is_not_claimed\": label claimed:* present","iid":17,"rule":"is_not_claimed"}
36 usine-usine | {"ts":"2026-08-27T16:01:26.185Z","level":"INFO","component":"supervisor","msg":"isAlive","activeWorkers
    ":1,"inFlightTickets":1}
37 usine-usine | {"ts":"2026-08-27T16:01:26.318Z","level":"DEBUG","component":"supervisor","msg":"Poll cycle #30","
    activeWorkers":1,"inFlightTickets":1}
38 usine-usine | {"ts":"2026-08-27T16:01:27.407Z","level":"DEBUG","component":"supervisor","msg":"found workflow: usine-
    PierreV::full for ticket id=17","id":17}
39 usine-usine | {"ts":"2026-08-27T16:01:27.409Z","level":"DEBUG","component":"supervisor","msg":"[ELIGIBILITY] #17 rejete
    par \"is_not_claimed\": label claimed:* present","iid":17,"rule":"is_not_claimed"}
40 usine-usine | {"ts":"2026-08-27T16:01:41.120Z","level":"INFO","component":"worker","msg":"Claude execution completed",
    "runId":"1017-17-20260827T155453Z","stepId":"code-full","exitCode":0,"outputLength":1268,"output":"Ticket #17
    implemented by the code-full subagent in two phases.\n\n**Source changes** (Escape key now deactivates the element
    picker):\n- Endpoint.enum.ts -- new ELEMENT_PICKER_CANCEL endpoint\n- content-script/.../ElementPicker.controller.
    ts -- cancelPicker handler calling the idempotent deactivatePickMode()\n- background/controllers/ElementPicker.
    controller.ts -- forwardCancelToTab forwarding to the active tab (mirrors forwardStartToTab)\n- sidebar/front/
    components/chat/Chat.vue -- isPickerActive ref, window keydown listener sending ELEMENT_PICKER_CANCEL on Escape
    only when active, cleaned up in new onBeforeUnmount\n\n**Tests** : new tests/components/chat/Chat.elementPicker.spec
    .ts -- 5 tests (START on activation, CANCEL on active Escape, no CANCEL when inactive, single-fire on repeated
    Escape, listener cleanup on unmount).\n\n**Limitation flagged by the subagent** : the worktree has no node_modules
    and npm install fails offline, so build/lint/type-check/tests were **not executed**. The spec mirrors the existing
    working Chat.spec.ts scaffold, but someone needs to run npm test in a provisioned environment to confirm green +
    coverage.\n\nReports written to .task/code-report.md, .task/test-report.md, .task/staged-files.txt.\n",
    stderrLength":0,"stderrlog":""}
41 usine-usine | {"ts":"2026-08-27T16:01:41.120Z","level":"DEBUG","component":"worker","msg":"Global attempt incremented
    ", "runId":"1017-17-20260827T155453Z","total":3}
42 usine-usine | {"ts":"2026-08-27T16:01:42.187Z","level":"DEBUG","component":"worker","msg":"Mechanical validation passed
    ", "runId":"1017-17-20260827T155453Z","stepId":"code-full"}
43 usine-usine | {"ts":"2026-08-27T16:01:42.187Z","level":"INFO","component":"worker","msg":"Step execution completed",
    "runId":"1017-17-20260827T155453Z","stepId":"code-full","success":true,"exitCode":0}
44 usine-usine | {"ts":"2026-08-27T16:01:42.552Z","level":"INFO","component":"worker","msg":"Progress committed", "runId
    ":"1017-17-20260827T155453Z","stepId":"code-full","sha":"ae722be9"}
45 usine-usine | {"ts":"2026-08-27T16:01:43.003Z","level":"WARN","component":"worker","msg":"Push failed (commit only)",
    "runId":"1017-17-20260827T155453Z","stepId":"code-full"}
46 usine-usine | {"ts":"2026-08-27T16:01:43.009Z","level":"INFO","component":"worker","msg":"Step notification", "runId
    ":"1017-17-20260827T155453Z","stepId":"code-full","status":"success","durationMs":219946,"nextStepId":"push"}
47 usine-usine | {"ts":"2026-08-27T16:01:43.011Z","level":"DEBUG","component":"worker","msg":"Execution log appended",
    "runId":"1017-17-20260827T155453Z","stepId":"code-full","totalEntries":3}
48 usine-usine | {"ts":"2026-08-27T16:01:43.037Z","level":"DEBUG","component":"worker","msg":"Manifest saved", "runId
    ":"1017-17-20260827T155453Z","path":"/repo/.worktrees/17/.task/manifest.json","currentStep":"push"}
49 usine-usine | {"ts":"2026-08-27T16:01:43.037Z","level":"INFO","component":"worker","msg":"=== Step starting", "runId
    ":"1017-17-20260827T155453Z","stepId":"push","index":3,"dispatchReason":"initial"}
50 usine-usine | {"ts":"2026-08-27T16:01:43.041Z","level":"DEBUG","component":"worker","msg":"Step execution starting",
    "runId":"1017-17-20260827T155453Z","stepId":"push","type":"script"}
51 usine-usine | {"ts":"2026-08-27T16:01:43.057Z","level":"INFO","component":"worker","msg":"Executing script step", "runId
    ":"1017-17-20260827T155453Z","stepId":"push","commandCount":4,"preview":"git rm -r --cached .task/ 2>/dev/null ||
    true; git add $(cat .task/staged-files.txt)"}
52 usine-usine | {"ts":"2026-08-27T16:01:43.604Z","level":"WARN","component":"worker","msg":"Script execution failed",
    "runId":"1017-17-20260827T155453Z","stepId":"push","exitCode":128,"output":"","error":"+ git rm -r --cached .task/\n
    + true\n++ cat .task/staged-files.txt\n+ git add src/common/shared/models/Endpoint.enum.ts src/content-script/
    content/controllers/ElementPicker.controller.ts src/background/controllers/ElementPicker.controller.ts src/sidebar/
    front/components/chat/Chat.vue tests/components/chat/Chat.elementPicker.spec.ts\n+ git diff --cached --quiet\n+ git
    push --force origin 17-fix-element-picker-escape-key-does-not-deactivate-element-pi\nfatal: could not read
    Username for 'https://git.oxyl.fr': No such device or address\n","failedCommand":"git push --force origin 17-fix-
    element-picker-escape-key-does-not-deactivate-element-pi"}
53 usine-usine | {"ts":"2026-08-27T16:01:43.606Z","level":"DEBUG","component":"worker","msg":"Global attempt incremented
    ", "runId":"1017-17-20260827T155453Z","total":4}
54 usine-usine | {"ts":"2026-08-27T16:01:43.607Z","level":"INFO","component":"worker","msg":"Step execution completed",
    "runId":"1017-17-20260827T155453Z","stepId":"push","success":false,"exitCode":128}
55 usine-usine | {"ts":"2026-08-27T16:01:43.610Z","level":"INFO","component":"worker","msg":"Failure classified", "runId
    ":"1017-17-20260827T155453Z","stepId":"push","category":"unknown","isRetryable":false}
56 usine-usine | {"ts":"2026-08-27T16:01:43.611Z","level":"INFO","component":"worker","msg":"Retrying step", "runId
    ":"1017-17-20260827T155453Z","stepId":"push","retryCount":1,"maxRetries":2}
57 usine-usine | {"ts":"2026-08-27T16:01:43.612Z","level":"DEBUG","component":"worker","msg":"Retry count incremented",
    "runId":"1017-17-20260827T155453Z","stepId":"push","count":1}
58 usine-usine | {"ts":"2026-08-27T16:01:43.630Z","level":"DEBUG","component":"worker","msg":"Manifest saved", "runId
    ":"1017-17-20260827T155453Z","path":"/repo/.worktrees/17/.task/manifest.json","currentStep":"push"}
59 usine-usine | {"ts":"2026-08-27T16:01:43.632Z","level":"INFO","component":"worker","msg":"Retrying step (1/2)", "runId

```

```

    "1017-17-20260827T155453Z", "stepId": "push"}
60 usine-usine | {"ts": "2026-08-27T16:01:43.636Z", "level": "INFO", "component": "worker", "msg": "=== Step starting", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (1/2)", "stepId": "push", "index": 3}
61 usine-usine | {"ts": "2026-08-27T16:01:43.636Z", "level": "DEBUG", "component": "worker", "msg": "Step execution starting", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (1/2)", "stepId": "push", "type": "script"}
62 usine-usine | {"ts": "2026-08-27T16:01:43.659Z", "level": "INFO", "component": "worker", "msg": "Executing script step", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (1/2)", "stepId": "push", "commandCount": 4, "preview": "git rm -r --cached .task/ 2>/dev/null || true; git add $(cat .task/staged-files.txt)"}
63 usine-usine | {"ts": "2026-08-27T16:01:44.052Z", "level": "WARN", "component": "worker", "msg": "Script execution failed", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (1/2)", "stepId": "push", "exitCode": 128, "output": "", "error": "+ git rm -r --cached .task/\n+ true\n++ cat .task/staged-files.txt\n+ git add src/common/shared/models/Endpoint.enum.ts src/content-script/content/controllers/ElementPicker.controller.ts src/background/controllers/ElementPicker.controller.ts src/sidebar/front/components/chat/Chat.vue tests/components/chat/Chat.elementPicker.spec.ts\n+ git diff --cached --quiet\n+ git push --force origin 17-fix-element-picker-escape-key-does-not-deactivate-element-pi\nfatal: could not read Username for 'https://git.oxyl.fr': No such device or address\n", "failedCommand": "git push --force origin 17-fix-element-picker-escape-key-does-not-deactivate-element-pi"}
64 usine-usine | {"ts": "2026-08-27T16:01:44.052Z", "level": "DEBUG", "component": "worker", "msg": "Global attempt incremented", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (1/2)", "total": 5}
65 usine-usine | {"ts": "2026-08-27T16:01:44.052Z", "level": "INFO", "component": "worker", "msg": "Step execution completed", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (1/2)", "stepId": "push", "success": false, "exitCode": 128}
66 usine-usine | {"ts": "2026-08-27T16:01:44.053Z", "level": "INFO", "component": "worker", "msg": "Failure classified", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (1/2)", "stepId": "push", "category": "unknown", "isRetryable": false}
67 usine-usine | {"ts": "2026-08-27T16:01:44.055Z", "level": "INFO", "component": "worker", "msg": "Retrying step", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (1/2)", "stepId": "push", "retryCount": 2, "maxRetries": 2}
68 usine-usine | {"ts": "2026-08-27T16:01:44.055Z", "level": "DEBUG", "component": "worker", "msg": "Retry count incremented", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (1/2)", "stepId": "push", "count": 2}
69 usine-usine | {"ts": "2026-08-27T16:01:44.069Z", "level": "DEBUG", "component": "worker", "msg": "Manifest saved", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (1/2)", "path": "/repo/.worktrees/17/.task/manifest.json", "currentStep": "push"}
70 usine-usine | {"ts": "2026-08-27T16:01:44.071Z", "level": "INFO", "component": "worker", "msg": "Retrying step (2/2)", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (1/2)", "stepId": "push"}
71 usine-usine | {"ts": "2026-08-27T16:01:44.072Z", "level": "INFO", "component": "worker", "msg": "=== Step starting", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (2/2)", "stepId": "push", "index": 3}
72 usine-usine | {"ts": "2026-08-27T16:01:44.072Z", "level": "DEBUG", "component": "worker", "msg": "Step execution starting", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (2/2)", "stepId": "push", "type": "script"}
73 usine-usine | {"ts": "2026-08-27T16:01:44.072Z", "level": "INFO", "component": "worker", "msg": "Executing script step", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (2/2)", "stepId": "push", "commandCount": 4, "preview": "git rm -r --cached .task/ 2>/dev/null || true; git add $(cat .task/staged-files.txt)"}
74 usine-usine | {"ts": "2026-08-27T16:01:44.322Z", "level": "WARN", "component": "worker", "msg": "Script execution failed", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (2/2)", "stepId": "push", "exitCode": 128, "output": "", "error": "+ git rm -r --cached .task/\n+ true\n++ cat .task/staged-files.txt\n+ git add src/common/shared/models/Endpoint.enum.ts src/content-script/content/controllers/ElementPicker.controller.ts src/background/controllers/ElementPicker.controller.ts src/sidebar/front/components/chat/Chat.vue tests/components/chat/Chat.elementPicker.spec.ts\n+ git diff --cached --quiet\n+ git push --force origin 17-fix-element-picker-escape-key-does-not-deactivate-element-pi\nfatal: could not read Username for 'https://git.oxyl.fr': No such device or address\n", "failedCommand": "git push --force origin 17-fix-element-picker-escape-key-does-not-deactivate-element-pi"}
75 usine-usine | {"ts": "2026-08-27T16:01:44.322Z", "level": "DEBUG", "component": "worker", "msg": "Global attempt incremented", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (2/2)", "total": 6}
76 usine-usine | {"ts": "2026-08-27T16:01:44.323Z", "level": "INFO", "component": "worker", "msg": "Step execution completed", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (2/2)", "stepId": "push", "success": false, "exitCode": 128}
77 usine-usine | {"ts": "2026-08-27T16:01:44.323Z", "level": "INFO", "component": "worker", "msg": "Failure classified", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (2/2)", "stepId": "push", "category": "unknown", "isRetryable": false}
78 usine-usine | {"ts": "2026-08-27T16:01:44.325Z", "level": "WARN", "component": "worker", "msg": "max_on_fail reached for script step, exiting", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (2/2)", "stepId": "push", "onFailCount": 0, "maxOnFail": null}
79 usine-usine | {"ts": "2026-08-27T16:01:44.326Z", "level": "INFO", "component": "worker", "msg": "[X] Step failed: Max retries reached for step push (2)", "runId": "1017-17-20260827T155453Z", "dispatchReason": "retry (2/2)", "stepId": "push", "exitCode": 1, "durationMs": 250}

```

Listing C.1 – Extrait des traces d'exécution de l'usine logicielle sur le Ticket #17

C.2 Analyse détaillée des jalons de l'exécution

L'examen pas-à-pas de ce journal d'audit confirme l'application effective des principes architecturaux détaillés dans les Chapitres 4 et 5 :

1. **Gestion distribuée de la concurrence (Superviseur) :** Les lignes émises par le composant supervisor démontrent l'efficacité de la règle d'exclusion mutuelle `is_not_claimed`. Alors que le worker exécute les sous-étapes dans son *Git Worktree* isolé, le superviseur détecte le ticket #17 à chaque cycle de scrutation (`Pollcycle\#20` à `\#30`) mais ignore son traitement car le label de verrouillage (`claimed: *`) est actif.
2. **Étanchéité et formalisation de la phase de planification (plan) :** L'étape `plan` instancie Claude CLI avec des arguments explicites (`--session-id`, `--allowed-tools`). L'agent formalise le plan dans `.task/plan.md` sans toucher au code source. Le moteur

applique ensuite la validation mécanique (`Mechanicalvalidationpassed`), enregistre l'état dans `manifest.json` et ordonnance le passage à l'étape suivante.

3. **Cloisonnement de l'outillage et traçabilité documentaire (`code-full`)** : L'étape d'implémentation ouvre l'outil d'édition (`Edit`) parmi les 7 outils autorisés. L'agent réalise les modifications sur 4 fichiers sources et rédige un fichier de test dédié (`Chat.elementPicker.spec.ts`). L'agent note explicitement l'environnement hermétique (*offline*) du conteneur empêchant l'installation de dépendances npm, consignant cette remarque dans `.task/code-report.md`. Un commit de progrès est scellé sous le SHA `ae722be9`.
4. **Gouvernance déterministe et disjoncteur face aux erreurs (`push`)** : Lors de l'étape finale scriptée, la commande `gitpush` échoue par absence d'identifiants interactifs (erreur Git standard en environnement d'intégration non configuré). Le moteur de résilience classe l'erreur, incrémente les compteurs de réessai (`retryCount` : 1 puis 2) et interrompt proprement la chaîne lorsque le seuil est atteint (`Maxretriesreached`), évitant tout blocage indéfini du conteneur.

D Vues d'écrans et Interfaces complémentaires de la plateforme NewroFactory

Cette annexe rassemble les captures d'écran des interfaces d'authentification, d'administration et de catalogue de la plateforme web *NewroFactory*, complétant les vues fonctionnelles présentées au Chapitre 3.

D.1 Authentification et Contrôle d'accès

La Figure D.1 présente la page d'accueil et d'authentification de la plateforme, garantissant un accès sécurisé et cloisonné selon les rôles attribués (ROLE_USER ou ROLE_ADMIN).



Se connecter

Email*
admin@oxyl.fr

Mot de passe*
.....

Se connecter

FIGURE D.1 – Page d'authentification et de connexion sécurisée à la plateforme NewroFactory

D.2 Tableau de bord général de supervision administrateur

La Figure D.2 illustre le tableau de bord d'accueil de l'administrateur et encadrant technique, consolidant les indicateurs clés (volume de compétences, nombre d'utilisateurs et de stagiaires actifs) et proposant des raccourcis vers les modules de gouvernance.

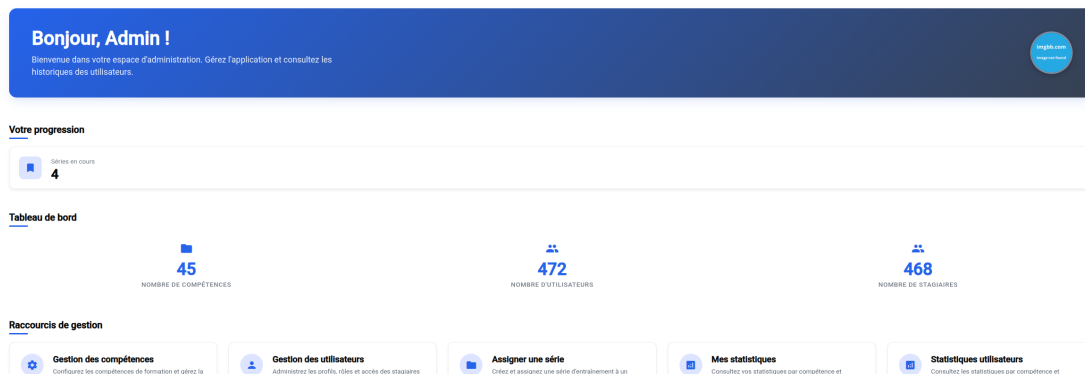


FIGURE D.2 – Tableau de bord général de l’administrateur avec métriques consolidées et raccourcis de gouvernance

D.3 Référentiel des compétences et modules de formation

La Figure D.3 présente le catalogue centralisé des compétences et certifications gérées sur la plateforme (Vue.js, Java OCA, Java OCP, Asynchronisme, ES6-ES11, Node CLI, Event Loop, File System, Process/OS, Spring Framework).

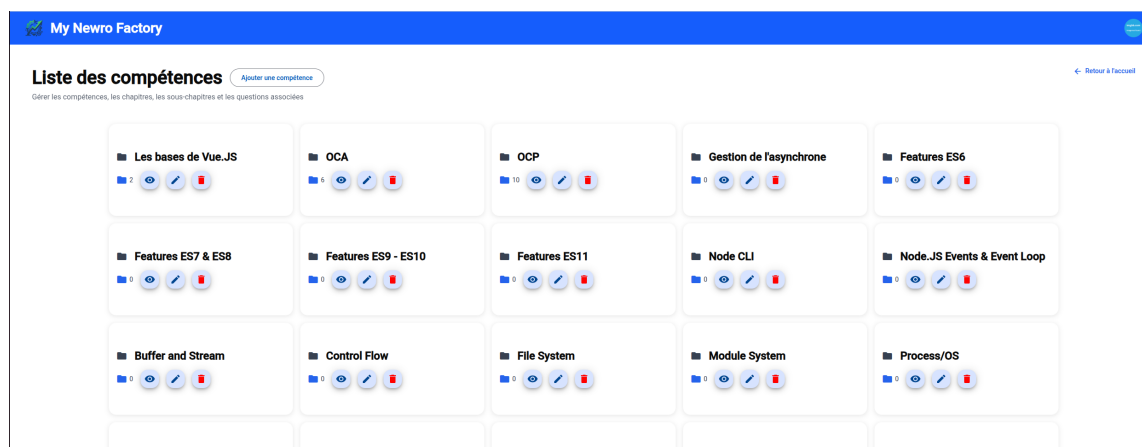


FIGURE D.3 – Catalogue et référentiel des compétences de formation et modules d’évaluation

D.4 Historique global et filtrage multicritères des séries

La Figure D.4 présente l’interface d’historique exhaustif des séries d’entraînement, dotée d’un curseur de filtrage interactif par taux de réussite (0% à 100%) et de mécanismes de tri chronologique.

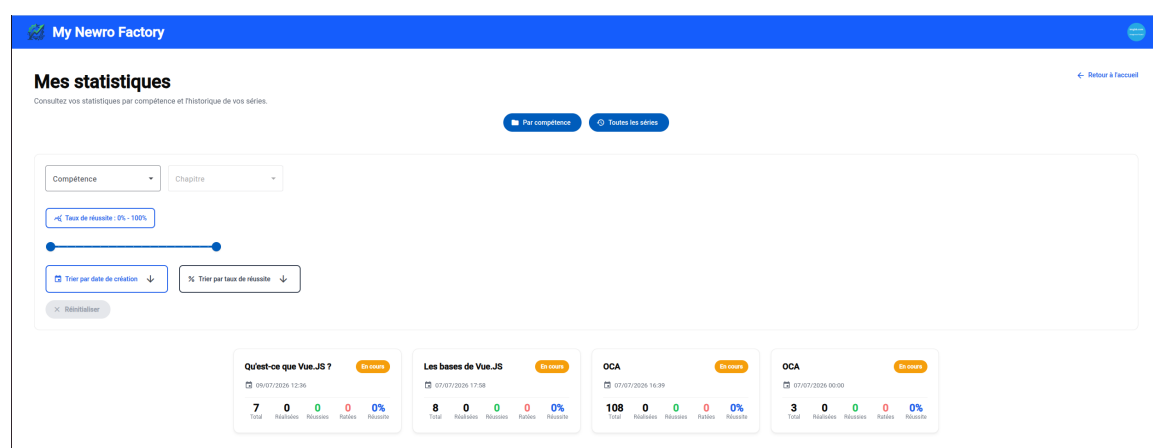


FIGURE D.4 – Interface de consultation et de filtrage dynamique de l'historique complet des séries

Résumé

Dans un secteur des Entreprises de Services du Numérique (ESN) en pleine mutation vers les engagements au forfait et confronté à l'avènement de l'intelligence artificielle générative, la rentabilité et l'excellence technique imposent une refonte des chaînes de production logicielle. Ce mémoire présente les travaux d'ingénierie menés au sein du Groupe Oxyl, articulés autour d'une double industrialisation : celle du **capital humain** et celle du **capital technique**.

Le premier volet formalise la standardisation des compétences à travers la conception d'une plateforme d'évaluation et de préparation aux certifications industrielles (Oracle OCP Java, AWS), architecturée en Spring Core et Angular. Le second volet porte sur le développement d'une **usine logicielle agentique** en TypeScript, orchestrant des agents d'IA autonomes pour la résolution automatisée de tickets de développement et de maintenance. Afin de prévenir tout verrouillage propriétaire (*vendor lock-in*), une couche d'abstraction logicielle fondée sur les patrons de conception (*Factory, Adapter, Strategy*) garantit une stricte indépendance vis-à-vis des modèles de langage (LLM) et des forges logicielles (GitLab, GitHub, Jira).

L'analyse démontre la synergie fondamentale entre ces deux piliers : loin de remplacer l'ingénieur, l'usine logicielle déplace sa charge cognitive vers l'architecture et l'arbitrage critique, exigeant une maîtrise théorique accrue pour superviser le code généré. Cette approche hybride optimise les coûts d'inférence, réduit le coût de non-qualité et pérennise l'avance technologique et l'attractivité commerciale de l'entreprise.

Mots-clés : Usine logicielle, Agents IA, Agnosticisme technologique, Génie logiciel.

Abstract

In the IT services and consulting industry (ESN), driven by an accelerated shift towards fixed-price engagements and the rise of generative artificial intelligence, sustained profitability and technical excellence require a comprehensive industrialization of software engineering workflows. This master's thesis presents the engineering achievements developed within Groupe Oxyl, structured around a dual-pillar strategy : the industrialization of **human capital** and **technical capital**.

The first pillar formalizes skills standardization through the design of a training and certification assessment platform (Oracle OCP Java, AWS), engineered with Spring Core and Angular. The second pillar focuses on the development of an **agentic software factory** in TypeScript, orchestrating autonomous AI agents to resolve development and maintenance tickets. To prevent vendor lock-in, an abstraction layer based on design patterns (*Factory, Adapter, Strategy*) ensures complete architectural independence from large language model (LLM) providers and software forges (GitLab, GitHub, Jira).

The analysis highlights the systemic synergy between both pillars : rather than replacing the engineer, the software factory elevates the cognitive workload toward architecture, specification, and critical review, demanding deeper theoretical mastery to supervise machine-generated code. This hybrid approach optimizes inference costs, mitigates the cost of non-quality, and secures a lasting technological edge and commercial attractiveness for the enterprise.

Keywords : Software Factory, AI Agents, Vendor Lock-in Prevention, Software Engineering.